SARABUN · ENGINEERING SERIES

เข้าใจระบบ แล้วพัฒนาต่อ

คู่มือนักพัฒนาระบบสารบรรณ

VERSION 1.4.1

สถาปัตยกรรม · TypeScript · ฐานข้อมูล
ตัวจัดหน้า · PDF / Word · งานพัสดุ
เพิ่มฟีเจอร์ · ทดสอบ · ส่งมอบโปรแกรม

คู่มือส่งต่องานและแบบเรียนจากโค้ดจริง
24 ตุลาคม 2569 · 44 บท
Source snapshot: ce43948

สารบัญ

กดชื่อบทเพื่อไปยังเนื้อหา เลขหน้าตรงกับหน้าในไฟล์ PDF

01	ขอบเขตของเล่มและวิธีใช้	4
02	แผนที่สถาปัตยกรรม: หนึ่งระบบ หลายชั้น	5
03	อ่านโฟลเดอร์ให้รู้ว่าควรเริ่มตรงไหน	6
04	ตั้งเครื่องพัฒนาโดยไม่แตะข้อมูลใช้งาน	7
05	เวอร์ชันและการตั้งค่าที่ต้องรู้	8
06	ตามรอยหนึ่งคำสั่ง: จากพิมพ์จนบันทึก	9
07	TypeScript: รู้ชนิดตอนเขียน ไม่ได้ตรวจข้อมูลแทนเรา	10
08	ใช้ Zod ให้ schema และ type ตรงกัน	11
09	Union และ narrowing: ลดเงื่อนไขที่ผิดชนิด	12
10	Generics และ utility types ที่ใช้ได้จริง	13
11	async, error และขอบเขต Client/Server	14
12	React + TypeScript: ฟอร์มที่ตามข้อมูลได้	15
13	สถานะหน้าจอ: dirty, busy, preview และ recovery	16
14	แบบข้อมูลและ SQLite: สิ่งที่ถูกเก็บจริง	17
15	Transaction, version และการกดซ้ำ	18
16	เปลี่ยน schema โดยยังเปิดข้อมูลเก่าได้	19
17	รูปแบบข้อความ: ไม่ใช่ HTML editor เต็มรูปแบบ	20
18	ตัวจัดหน้า: จาก block ไปเป็นกระดาษ A4	21
19	เรขาคณิตเอกสารและบทเรียนจากรุ่น 1.4.1	22
20	สร้าง PDF: ลำดับและการวินิจฉัย	23
21	Word: ใช้โครงสร้างแก้ไขได้ ไม่ใช่ถ่ายภาพทั้งฉบับ	24
22	เอกสารแนบสามชนิดที่ห้ามสับสน	25

อ่านเพื่อรับงาน: บท 1-6, 14-16, 30-31, 38-44

อ่านเพื่อฝึกเขียน: บท 7-13, 33-34, 41

สารบัญ (ต่อ)

กดชื่อบทเพื่อไปยังเนื้อหา เลขหน้าตรงกับหน้าในไฟล์ PDF

23	ภาพลายเซ็นและข้อความลอย: พิกัดต้องไม่ผูกกับซูม	26
24	คลังคำไทย: ตรวจคำกับกันแยกคำทำงานคนละส่วน	27
25	ทะเบียนรับ-ส่ง: เลขทะเบียนและร่างเป็นคนละอย่าง	28
26	พัสดุ: state machine และความถูกต้องของยอด	29
27	แบบฟอร์มพัสดุ 8 แบบ: นิยามข้อมูลนำหน้ารูปแบบ	30
28	รายงานและตัวชี้วัด: รู้ว่าตัวเลขหมายถึงอะไร	31
29	โมดูลอื่นที่ต้องรู้ก่อนเปลี่ยนระบบร่วม	32
30	สำรอง กู้คืน และการเก็บรักษาข้อมูล	33
31	Drive และสิทธิ์: ขอบเขตของเครื่องเดียว	34
32	ปรับ UX/UI โดยไม่ทำให้แบบพิมพ์เปลี่ยน	35
33	Workshop: เพิ่มผู้ประสานงานในหนังสือภายนอก	36
34	Workshop: พิสูจน์ว่าฟิลด์ใหม่ทำงานครบ	37
35	เพิ่มแบบหนังสือหรือเปลี่ยนรูปแบบการพิมพ์	38
36	ลดฟีเจอร์หรือซ่อนงาน: ไม่ทำข้อมูลเก่าหาย	39
37	เพิ่มโมดูลใหม่โดยใช้รูปแบบเดิมของระบบ	40
38	ทดสอบเป็นชั้นและใช้ข้อมูลแยกเสมอ	41
39	ตารางแก้ปัญหา: ไล่จากอาการไปหลักฐาน	42
40	Build, แจกโปรแกรม และ rollback	43
41	เส้นทางเรียนรู้และแบบฝึกพร้อมเกณฑ์ผ่าน	44
42	หนี้ทางเทคนิคและลำดับปรับปรุงที่เหมาะสม	45
43	แหล่งอ้างอิงและดัชนีค้นเร็ว	46
44	เช็กลิสต์รับช่วงและนิยามงานเสร็จ	47

อ่านเพื่อรับงาน: บท 1-6, 14-16, 30-31, 38-44

อ่านเพื่อฝึกเขียน: บท 7-13, 33-34, 41

01

ขอบเขตของเล่มและวิธีใช้

คู่มือนี้อธิบายระบบสารบรรณ **เวอร์ชัน 1.4.1** ณ 24 ตุลาคม 2569 โดยอ่านจากซอร์ส commit `ce439486e3c824eecc31f496279bc055ad471335` ของ KTTNDev/Administater-system ใช้เป็นคู่มือส่งต่องานและแบบเรียนจากโปรเจกต์จริง ข้อมูลตัวอย่างในเล่มเป็นข้อมูลสมมติ ไม่ดึงเอกสารหรือความลับของผู้ใช้มาประกอบ

ระบบที่กำลังส่งต่อ

แอป Next.js ที่เปิดเป็นหน้าต่าง Edge/Chrome บน Windows x64 ใช้ Node.js และ SQLite ภายในเครื่องเดียว มีหนังสือราชการ 13 แบบ ทะเบียนรับ-ส่ง งานพัสดุ แบบฟอร์มพัสดุ 8 แบบ งานธุรการ โอนงบ ปฏิทิน ห้องประชุม และคลังกฎหมาย

หน้าต่างโปรแกรมกับเบราว์เซอร์ใช้เว็บแอปชุดเดียวกัน โปรแกรมแบบ desktop ไม่ใช่ Electron และไม่ใช่ไฟล์ .exe ที่แยกตรรกะธุรกิจออกอีกชุด

อ่านตามเป้าหมาย

- **รับช่วงดูแล:** อ่านบท 1-6, 14-16, 30-31 และ 38-44 ก่อนแก้ข้อมูลจริง
- **เรียน TypeScript:** อ่านบท 7-13 แล้วทำแบบฝึกในบท 33-34 และ 41
- **ปรับฟอร์ม/งานพิมพ์:** อ่านบท 17-24 และ 35 ควบคู่ไฟล์ที่อ้างอิงท้ายบท
- **เพิ่มระบบงาน:** อ่านบท 25-29 และ 37 โดยรักษาการสำรองและการทดสอบไว้ด้วย

แยกสิ่งที่มีจริงกับสิ่งที่เสนอ

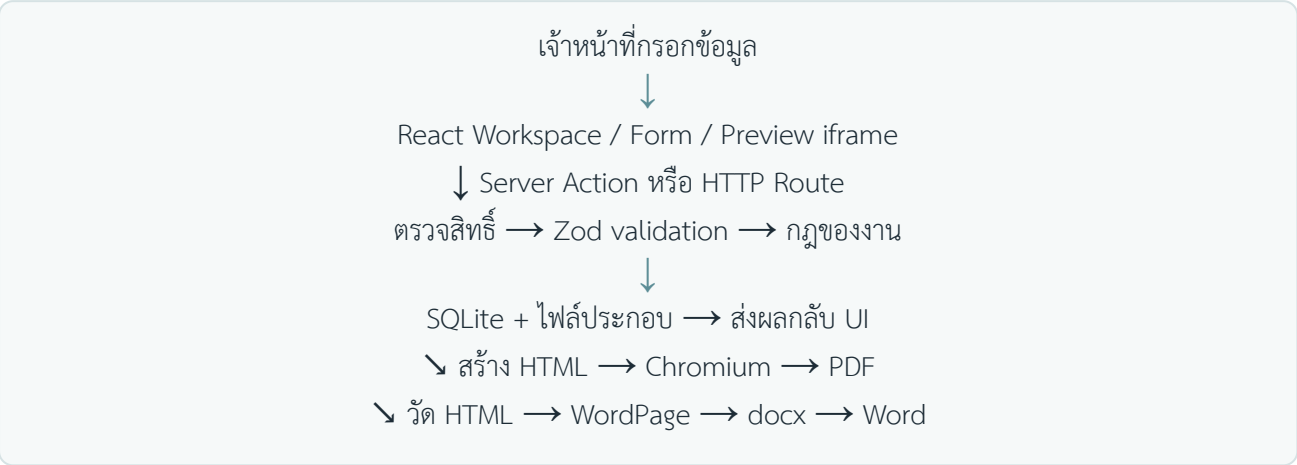
คำว่า **ปัจจุบัน** หมายถึงพบในโค้ดรุ่นนี้ ส่วน **ตัวอย่างฝึก** และ **แนวทางต่อยอด** ยังไม่ใช่ฟีเจอร์ที่ติดตั้งแล้ว ตัวอย่างย่อ อาจละ import หรือ UI รอบข้าง โดยระบุไว้ในเนื้อหา

ระบบออกแบบเพื่อผู้ใช้คนเดียวในเครื่องเดียว ชื่อผู้ปฏิบัติงานที่กรอกและภาพลายเซ็นเป็นหลักฐานประกอบงาน ไม่ใช่ระบบยืนยันตัวบุคคลหรือการลงนามดิจิทัลด้วยใบรับรอง การขยายเป็นหลายผู้ใช้ต้องออกแบบสิทธิ์และการตรวจสอบใหม่

หลักฐานรุ่น: `package.json`, `CHANGELOG.md`, `docs/STATUS-1.4.0.md` และ commit ที่ระบุข้างต้น README บางส่วนยังใช้ชื่อรุ่นเก่า จึงไม่ใช้เป็นแหล่งยืนยันเลขรุ่นเพียงแห่งเดียว

02

แผนที่สถาปัตยกรรม: หนึ่งระบบ หลายชั้น



หน้าที่ของแต่ละชั้น

ชั้น	รับผิดชอบ	ไม่ควรย้ายเข้าไป
components	รับค่า แสดงผล จัดสถานะหน้าจอ	SQL, OAuth secret
app/*-actions.ts	รับคำสั่ง ตรวจผู้ใช้ ตรวจ input	การจัดหน้า A4 ทุกบรรทัด
lib/* domain	schema กฎ คำนวณ แปลงข้อมูล	การกดปุ่มและ focus
lib/*-db.ts	transaction อ่านเขียน ตรวจ version	state ของ React
renderer/export	รูปแบบเอกสารและไฟล์ส่งออก	การตัดยอดสต็อก
scripts	build เปิดโปรแกรม สำรอง อัปเดต	กฎธุรกิจชุดใหม่ที่ซ้ำกับ lib

เส้นแบ่งที่สำคัญ

`src/app/page.tsx` ทำงานฝั่ง server โหลดข้อมูลเริ่มต้นแล้วส่งให้ `Workspace` ซึ่งเป็น Client Component การนำเข้า `db.ts` จาก component ฝั่ง client จะข้ามขอบเขตที่ผิด ใช้ Server Action เป็นทางผ่านแทน

HTML เอกสารอยู่ใน iframe ของตัวเอง จึงมี CSS ของเอกสารแยกจาก CSS หน้าเว็บ การเปลี่ยนสี sidebar ไม่ควรเปลี่ยนหมึกที่พิมพ์ลงกระดาษ

อ่านโค้ด: `src/app/page.tsx`, `src/components/workspace.tsx`, `src/app/actions.ts`, `src/lib/db.ts`, `src/lib/render-document.ts`

03

อ่านโฟลเดอร์ให้รู้ว่าควรเริ่มตรงไหน

ตำแหน่ง	ใช้ทำอะไร
src/app	หน้า Next.js, Server Actions, API routes และ CSS หลัก
src/components	ฟอร์ม ตัวอย่างเอกสาร และพื้นที่ทำงานแต่ละโมดูล
src/lib	แบบข้อมูล กฎธุรกิจ DB renderer และ export
scripts	เริ่มระบบ build desktop ทดสอบรุ่น สำรองและอัปเดต
tests/*.test.ts	ทดสอบฟังก์ชัน DB และโครงสร้างเอกสารด้วย Vitest
tests/e2e	ทดสอบการใช้งาน/จัดหน้าด้วย Playwright
public/fonts	TH Sarabun New ปกติและตัวหนา
public/assets	ครุฑ ตราร่าง และตราสำเนาคู่ฉบับ
public/dictionaries	ฐานคำไทยและข้อมูลใบอนุญาต
docs	คู่มือเฉพาะงาน สถานะ และผลตรวจย้อนหลัง

ซอร์สไม่ใช่ข้อมูลผู้ใช้

`data/` เก็บฐานข้อมูลและไฟล์อ้างอิง ส่วน `output/` เก็บผลส่งออก log และผลทดสอบ ทั้งสองไม่อยู่ใน Git ตาม .gitignore อย่าใช้การ push Git เป็นการสำรองเอกสารผู้ใช้

`.next-workspace/` คือผล build/dev ทั่วไป; `.next-desktop/` คือผล build สำหรับแจกโปรแกรม; `.desktop/versions/` คือชุดที่เปิดใช้งานจริงแบบแยกรุ่น และ `.desktop/current.json` ชี้รุ่นปัจจุบัน

`releases/` เป็นแพ็กเกจที่แจกได้ แต่บางแบบอาจมีข้อมูลส่วนตัว ต้องแยก Clean กับ Personal ก่อนส่งให้คนอื่น

วิธีค้นหาอย่างมีเป้าหมาย

POWERSHELL

```
rg -n 'saveDocument|persist' src/components/workspace.tsx
rg -n 'documentHtml|__paginationDone' src/lib
rg -n 'CREATE TABLE|transaction' src/lib
rg --files tests/e2e
```

ค้นจากชื่อปุ่ม ⟶ handler ⟶ action ⟶ lib จะเร็วกว่าอ่านไฟล์ใหญ่ทั้งหมดตั้งแต่ต้น

อ่านโค้ด: `.gitignore`, `next.config.mjs`, `scripts/build-desktop.mjs`

04

ตั้งเครื่องพัฒนาโดยไม่แตะข้อมูลใช้งาน

ใช้ checkout แยกสำหรับฝึกและพัฒนา

เครื่องที่จัดทำคู่มือนี้ใช้ซอร์สจริงที่ `C:/Users/rawai/source/sarabun` ส่วนประวัติเก่าเคยใช้ D:/sarabun ผู้รับช่วงควรตรวจโฟลเดอร์จริงของตน ไม่คัดลอกพาธจากข้อความเก่าโดยอัตโนมัติ

เตรียม Git, Node.js 22.13 ขึ้นไป, npm และ Chrome/Edge ที่เข้ากับชุดนี้ แนะนำเริ่มจาก Node 22.13.1 ที่ใช้กับรุ่นส่งมอบ ก่อนอัปเกรดให้ทดสอบ node:sqlite และแพ็กเกจ desktop ใหม่

```powershell
git clone https://github.com/KTTNDev/Administater-system.git
cd Administater-system
# ศึกษาซอร์สรุ่นที่คู่มือนี้อ้างอิง
# งานใหม่ควรสร้าง branch จากรุ่นที่ทีมตกลง
npm ci
Copy-Item .env.example .env.local
node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"
```

นำค่าสุ่มไปใส่ SESSION_SECRET ของ **ชุดพัฒนาใหม่เท่านั้น** อย่าเปลี่ยนคีย์เดิมของชุดใช้งาน เพราะใช้เข้ารหัสข้อมูลเชื่อม Drive ด้วย

ตั้งค่าเฉพาะชุดฝึก

```ini
APP_URL=http://127.0.0.1:3200
DATABASE_PATH=./data/dev-only.sqlite
SARABUN_REQUIRE_LOGIN=false
LOCAL_DEMO=true
```

```powershell
npx next dev --hostname 127.0.0.1 --port 3200
```

คำสั่งนี้ใช้พอร์ต 3200 โดยตรง เพราะ `npm run dev` ของรุ่นนี้ระบุพอร์ต 3000 ไว้แล้ว หากใช้พอร์ต 3000 ต้องหยุด desktop ที่ครองพอร์ตก่อน ห้ามเปิด dev บนฐานข้อมูลใช้งานเพื่อทดลองกฎหรือ migration

`npm ci` ใช้ package-lock.json และอาจต้องเข้าถึงอินเทอร์เน็ต ติดตั้งแล้วเปิดหน้าทดสอบ สร้างร่างสมมติ บันทึก เปิดใหม่ และส่งออก PDF ก่อนเริ่มแก้ฟีเจอร์

05

เวอร์ชันและการตั้งค่าที่ต้องรู้

รายการในรุ่น 1.4.1	เวอร์ชัน / ค่า
Next.js / React	16.4.2 / 19.2.8
TypeScript / Zod	5.9.3 / 4.5.4
docx / Playwright	9.10.0 / 1.63.0
Vitest	5.0.5
tsconfig	strict, noEmit, ES2022, moduleResolution: bundler
alias	@/* ⟶ src/*

ตัวแปรแวดล้อมที่กระทบระบบ

ตัวแปร	ความหมายและข้อควรระวัง
DATABASE_PATH	พาธ DB; ค่าเริ่มต้น data/sarabun.sqlite
SARABUN_SOURCE_DIR	รากข้อมูลของ desktop/test ที่แยกจาก runtime
APP_URL	origin ของระบบ ต้องตรง Google callback
SARABUN_REQUIRE_LOGIN	true จึงเปิดหน้ารหัสผ่าน
SESSION_SECRET	session และคีย์เข้ารหัส Drive ต้องรักษาค่าเดิม
CHROMIUM_EXECUTABLE_PATH	ระบุ Chrome/Edge สำหรับ export และทดสอบ
GOOGLE_CLIENT_ID / SECRET	OAuth ฝั่ง server ห้ามส่งลง browser
SARABUN_BUILD	desktop ใช้ standalone build

`APP_URL` ไม่ได้เปลี่ยนพอร์ต listener ของคำสั่ง dev ให้เอง ต้องตั้งพอร์ตคำสั่งให้ตรงกัน ตัวแปร `LOCAL_DEMO` ใช้เฉพาะ development ไม่ใช่เครื่องมือจัดการสิทธิ์หลายบทบาท

อ่าน `.env.example` เพื่อรู้ชื่อคีย์ แต่ไม่ใส่ค่าจริงลงคู่มือ issue หรือ Git การแชร์ log ต้องตรวจ token และข้อมูลเอกสารด้วย

Next.js รุ่นนี้มีคู่มือใน `node_modules/next/dist/docs/` และ AGENTS.md กำหนดให้อ่านส่วนที่เกี่ยวข้องก่อนแก้ อย่าปรับ API ตามตัวอย่าง Next รุ่นเก่าโดยไม่ตรวจ

อ่านโค้ด: `package.json`, `tsconfig.json`, `next.config.mjs`, `src/lib/auth.ts`

06

ตามรอยหนึ่งคำสั่ง: จากพิมพ์จนบันทึก

กรณีเปลี่ยนชื่อเรื่องแล้วกดบันทึก

1. `DocumentForm` รับ `Draft` และ callback เปลี่ยนบางฟิลด์
2. `Workspace` เปลี่ยน state ของ draft; ตัวอย่างอ่านข้อมูลที่เตรียมไว้สำหรับ preview
3. `persist()` เก็บ snapshot ของข้อความขณะส่ง แล้วเรียก `saveDocument(draft, id, version)`
4. Server Action เรียก `requireUser()` และ `documentSchema.parse(input)`
5. ตรวจ metadata ของไฟล์แนบ/รูป และเงื่อนไขเมื่อเลือกสถานะ reviewed
6. `writeDraft()` เปิด transaction ตรวจเลขและ version แล้วบันทึก JSON กับ audit
7. UI รับ SavedDraft ใหม่ ปรับ baseline และโหลดรายการล่าสุด

ทำไมต้องเก็บ snapshot ตอนกด

ผู้ใช้พิมพ์ต่อได้ระหว่างรอเครือข่าย หากตอบกลับแล้วนำ saved เก่าทับ draft ทันที ข้อความที่พิมพ์ระหว่างรอจะหาย โค้ดจึงตรวจว่าข้อมูลปัจจุบันยังตรงกับสิ่งที่ส่งหรือไม่ก่อนนำ canonical draft กลับใส่ state

```ts
// ย่อจากแนวทาง persist() ใน Workspace
const submitted = JSON.stringify(draft);
const result = await saveDocument(draft, saved?.id, saved?.version);
if (!result.ok) return;
const canonical = documentSchema.parse(result.data);
setDraft(current =>
  JSON.stringify(current) === submitted ? canonical : current
);
```

`baseline` เป็นข้อมูลที่บันทึกสำเร็จล่าสุด ใช้เทียบว่ามีการแก้ค้างอยู่ การโหลดตัวอย่างใหม่ไม่เท่ากับบันทึกร่าง และสำเนากู้คืนก็ไม่ใช่ SavedDraft ที่มีเลขทะเบียน

ลองตามโค้ด: ตั้ง breakpoint ก่อน `writeDraft()` แล้วตรวจ id/version ในชุดฝึก โดยไม่ log เนื้อหาเอกสารจริง

อ่านโค้ด: `src/components/workspace.tsx` ฟังก์ชัน persist, `src/app/actions.ts` ฟังก์ชัน saveDocument, `src/lib/db.ts` ฟังก์ชัน writeDraft

07

TypeScript: รู้ชนิดตอนเขียน ไม่ได้ตรวจข้อมูลแทนเรา

JavaScript เป็นสิ่งที่รันจริง TypeScript เพิ่มการตรวจชนิดระหว่างพัฒนา เช่น ส่ง number ไปยังฟังก์ชันที่ต้องการ string แล้วเตือนก่อนรัน ชนิดส่วนใหญ่ถูกลบเมื่อสร้าง JavaScript

```ts
// ตัวอย่างฝึก: types ไม่ได้เปลี่ยนค่าข้อมูล
function documentLabel(subject: string, version: number): string {
  return subject + ' / v' + version;
}
const label = documentLabel('Example', 1);
// documentLabel('Example', '1'); // compiler เตือน
```

ความหมายของรูปแบบที่พบบ่อย

รูปแบบ	ความหมายในโปรเจกต์
string / number / boolean	ข้อความ จำนวน และตัวเลือกจริง/เท็จ
Draft[]	รายการร่างหลายรายการ
id?: string	อาจไม่ได้ส่ง id มา
string | null	ไม่มีค่าอย่างชัดเจน เช่น driveFileId
Draft & { id: string }	ต้องมีข้อมูล Draft และ id
import type	นำเข้าเฉพาะชนิด ไม่ใช่ค่าที่ใช้ตอนรัน

unknown เหมาะกับขอบเขตรับข้อมูล

```ts
const input: unknown = JSON.parse(rawText);
// input.subject ใช้ตรง ๆ ไม่ได้ เพราะยังไม่รู้โครงสร้าง
const draft = documentSchema.parse(input);
console.log(draft.subject); // ผ่านการตรวจ runtime แล้ว
```

`input as Draft` เป็นการบอก compiler ให้เชื่อ ไม่ได้ตรวจว่าข้อมูลถูกต้อง เช่น JSON เก่าขาดฟิลด์ หรือผู้ใช้ดัดแปลง request ดังนั้น action ใช้ unknown แล้ว parse เสมอ อย่าแก้ error ด้วย any/as โดยยังไม่รู้สาเหตุ

อ่านโค้ด: `src/lib/document.ts`, `src/app/actions.ts` • **อ่านต่อ:** TypeScript Everyday Types [R1]

08

ใช้ Zod ให้ schema และ type ตรงกัน

`documentSchema` เป็นสัญญาของร่างทั้งฝั่ง UI การบันทึก และการเปิดข้อมูลเก่า ส่วน `Draft` ได้มาจาก schema เดียวกัน จึงลดปัญหาประกาศ type แล้วลืมแก้ตัวตรวจข้อมูล

```ts
// ตัวอย่างฝึก แยกจาก schema จริงเพื่อให้เห็นหลักการ
import { z } from 'zod';
const noteSchema = z.object({
  subject: z.string().trim().min(1).max(300),
  contact: z.string().max(200).default(''),
  reviewed: z.boolean().default(false),
});
type Note = z.infer<typeof noteSchema>;
const note = noteSchema.parse({ subject: 'Example' });
// note.contact === '' และ note.reviewed === false
```

parse กับ safeParse

`parse` คืนข้อมูลที่ผ่านเงื่อนไขหรือโยน ZodError เหมาะกับ action ที่มี try/catch อยู่แล้ว `safeParse` คืนผล success/data หรือ success/error เหมาะกับการตรวจและแสดงข้อผิดพลาดเฉพาะจุด

```ts
const result = noteSchema.safeParse(input);
if (!result.success) {
  return { ok: false as const, error: result.error.issues[0].message };
}
return { ok: true as const, data: result.data };
```

default ช่วยเรื่องความเข้ากันได้ แต่ต้องมีความหมาย

เพิ่มฟิลด์ใหม่ที่มี `.default('')` ทำให้ร่างเก่าเปิดได้เมื่อไม่มีฟิลด์นั้น แต่ถ้าเปลี่ยนความหมายของข้อมูลเดิม เช่น วัน พ.ศ. กลายเป็น ค.ศ. ต้องมี migration จริง default ไม่ได้แปลงค่าที่มีอยู่แล้ว

ในรุ่นนี้ subject ต้องไม่ว่างเพื่อบันทึกร่าง ส่วน `reviewIssues()` ตรวจความครบก่อน reviewed แยกจาก schema: “เป็นข้อมูลถูกชนิด” กับ “พร้อมออกหนังสือ” เป็นคนละข้อกำหนด

อ่านโค้ด: `src/lib/document.ts` documentSchema/reviewIssues, `src/app/actions.ts` • **อ่านต่อ:** Zod Basics [R3]

09

Union และ narrowing: ลดเงื่อนไขที่ผิดชนิด

ภาพลอยกับข้อความลอยมีฟิลด์ร่วม เช่น page/xMm/yMm แต่ไม่ควรบังคับให้ทุกชนิดมีทั้ง text และ image ระบบจึงใช้ discriminated union โดยให้ `kind` บอกประเภท

```ts
// ตัวอย่างฝึก ย่อรูปแบบจาก floatingItems
 type Floating =
  | { kind: 'text'; text: string; fontPt: number }
  | { kind: 'image'; imageId: string; widthMm: number };

function describe(item: Floating): string {
  if (item.kind === 'text') return item.text;
  return item.imageId;
}
```

หลังตรวจ kind แล้ว TypeScript รู้ว่าฟิลด์ใดใช้ได้ การเข้าถึง item.text โดยไม่แยกประเภทจะถูกเตือน ซึ่งช่วยเมื่อต่อไปเพิ่ม QR หรือชนิดใหม่

ผล action ก็ใช้รูปแบบเดียวกัน

```ts
type Result<T> =
  | { ok: true; data: T }
  | { ok: false; error: string };

function message(result: Result<{ subject: string }>) {
  if (!result.ok) return result.error;
  return result.data.subject;
}
```

`as const` บน `ok:true` ในโค้ดจริงรักษาชนิด literal true แทนการขยายเป็น boolean ทำให้เงื่อนไข `if (!result.ok)` แยกแขนงได้แม่นยำ

ตรวจว่ารองรับทุกสถานะ

เมื่อใช้ switch สามารถเพิ่ม `const neverValue: never = item` ในแขนงสุดท้ายเพื่อให้ compiler เตือนเมื่อเพิ่มประเภทแต่ยังไม่ได้จัดการ วิธีนี้เหมาะกับส่วนที่ต้องครอบคลุมทุกชนิดจริง ไม่เหมาะกับส่วนที่ตั้งใจแสดงเฉพาะบางชนิด

อ่านโค้ด: `src/lib/document.ts` floatingItems, `src/app/inventory-actions.ts` command • **อ่านต่อ:** TypeScript Narrowing [R2]

10

Generics และ utility types ที่ใช้ได้จริง

Generic คือการเก็บความสัมพันธ์ของชนิดไว้ เช่น ฟังก์ชันรับ Draft แล้วควรคืน Draft ไม่ใช่ unknown ชนิด T ไม่ใช่ค่าที่เกิดขึ้นตอน runtime

```ts
// ตัวอย่างฝึก: เปลี่ยนฟิลด์โดยยังตรวจชนิดของค่า
function setField<T, K extends keyof T>(
  current: T, key: K, value: T[K]
): T {
  return { ...current, [key]: value };
}
const next = setField(newDraft(), 'memoGuides', false);
// setField(newDraft(), 'memoGuides', 'false'); // ผิดชนิด
```

`keyof T` คือชุดชื่อฟิลด์; `T[K]` คือชนิดค่าของฟิลด์นั้น ต่างจากการรับ key:string,value:any ซึ่งไม่ช่วยป้องกันพิมพ์ชื่อฟิลด์ผิด

Type utility	การใช้งาน
Partial<Draft>	patch บางฟิลด์จากฟอร์ม ไม่ใช่ร่างที่สมบูรณ์
Pick<Draft, 'subject' | 'date'>	เลือกเฉพาะข้อมูลที่ component ต้องใช้
Omit<MovementInput, 'lines'>	ยกเว้น lines เพื่อแทนด้วยชนิดที่มี snapshot
Draft['floatingItems'][number]	ชนิดสมาชิกหนึ่งรายการใน array
Record<string, string>	map id ของรูปไปยัง data URL

ตัวอย่างจาก transaction ในระบบ

`transaction<T>(operation: () => T): () => T` รับ callback แล้วคืนฟังก์ชันที่เมื่อเรียกจะเริ่ม transaction และคืนชนิดเดียวกับ callback จึงได้ SavedDraft กลับออกมาโดยไม่ต้อง cast

สำคัญ: ตัว wrapper นี้ทำงาน synchronous ถ้าส่ง async callback จะได้ Promise และ COMMIT ก่อนงาน await จบ ห้ามนำ network/Drive call เข้า transaction นี้ แม้ TypeScript จะยอมให้ T เป็น Promise ก็ตาม

อ่านโค้ด: `src/lib/db.ts` SQLiteDatabase.transaction, `src/lib/inventory.ts` Movement • **อ่านต่อ:** Generics [R4], Utility Types [R5]

11

async, error และขอบเขต Client/Server

ในรุ่นนี้ `page.tsx` เป็น async Server Component และรับ searchParams แบบ Promise ต้อง await ก่อนอ่าน ส่วน action ใช้ `'use server'` และฟังก์ชัน async แม้ SQL ภายในจะเป็น synchronous

```ts
// ตัวอย่างฝึก: โครง action ที่ควรใช้กับคำสั่งใหม่
'use server';
import { requireUser } from '@/lib/auth';

export async function execute(input: unknown) {
  try {
    await requireUser();
    // 1. schema.parse(input)
    // 2. call a synchronous domain/DB function
    return { ok: true as const };
  } catch (error: unknown) {
    return {
      ok: false as const,
      error: error instanceof Error ? error.message : 'Failed',
    };
  }
}
```

นี่เป็นโครงอธิบาย ยังไม่มีการบันทึกจริง อย่าคัดลอกแล้วถือว่าคำสั่งทำงานครบ action ต้องตรวจ input และกฎของงานด้วย

รอสิ่งใด ปิดสิ่งใด

งาน renderPdf/renderWord เปิด Chromium แล้วปิดใน finally เพื่อให้ปิด browser แม้ export ล้มเหลว รูปแบบนี้ควรใช้กับ resource ที่มีอายุ เช่น connection หรือไฟล์ที่เปิดค้าง

การจับ error ไม่ควรทำให้ UI เข้าใจว่าสำเร็จ ให้คืนสถานะชัดเจน ปลด busy ใน finally ฝั่ง client และแสดงข้อความที่ทำต่อได้ เช่น เปิดร่างล่าสุดเมื่อ version ขัดกัน

อย่าส่งรายละเอียด path ภายใน token หรือ stack trace ให้ผู้ใช้ผ่าน API ใช้ log สำหรับนักพัฒนา แต่ต้องไม่ log รหัสผ่านหรือเอกสารทั้งฉบับตามปกติ

อ่านโค้ด: `src/app/actions.ts`, `src/lib/pdf.ts`, `src/app/api/documents/[id]/word/route.ts` • คู่มือ Next ที่ติดตั้ง: Server and Client Components / Mutating Data

12

React + TypeScript: ฟอร์มที่ตามข้อมูลได้

Component แสดงผลจาก props และ state `DocumentForm` รับ Draft พร้อม callback เปลี่ยนค่า ส่วน `Workspace` ถือ state กลาง การแยกเช่นนี้ทำให้ฟอร์มไม่ได้บันทึกฐานข้อมูลด้วยตัวเองทุกช่อง

TSX

```tsx
// ตัวอย่างฝึก: component ที่แก้เพียงข้อมูลการติดต่อ
import type { Draft } from '@/lib/document';
type Props = {
  draft: Draft;
  onChange: (patch: Partial<Draft>) => void;
};
export function ContactField({ draft, onChange }: Props) {
  return (
    <label>
      Contact
      <input value={draft.phone}
        onChange={e => onChange({ phone: e.target.value })} />
    </label>
  );
}
```

เปลี่ยน object แทนการแก้ของเดิม

ใช้ `setDraft(previous => ({ ...previous, ...patch }))` ไม่ใช้ `draft.phone = value` เพราะ React ต้องรู้ว่ามี state ใหม่ การแก้ array เช่น opinions ให้สร้าง array ใหม่และคง id ของรายการที่ต้องรักษาตัวตน

`useRef` เหมาะกับ DOM เช่น textarea และค่าที่ไม่ต้อง render ทุกครั้ง; `useState` เหมาะกับสิ่งที่แสดงบนหน้าจอ; `useEffect` ใช้ประสานกับสิ่งภายนอก เช่น event listener หรือสร้าง preview แล้วต้องมี cleanup

การพิมพ์ต้องไม่เสีย focus

อย่าสร้าง key ใหม่ทุกครั้งบน input เพราะจะถอดและสร้าง DOM ใหม่ ทำให้ตำแหน่ง cursor หาย ส่วน iframe ใช้ frameKey เพื่อจงใจเริ่มตัวอย่างใหม่เมื่อจำเป็น เป็นคนละเหตุผลกับ input

การซ่อนช่องไม่ควรลบค่าที่กรอกโดยอัตโนมัติ ถ้าการเปลี่ยนชนิดเอกสารต้องล้างข้อมูล ให้ระบุนโยบายชัดและทดสอบกลับไปกลับมาด้วย

อ่านโค้ด: `src/components/document-form.tsx`, `src/components/body-editor.tsx`, `src/components/workspace.tsx`

13

สถานะหน้าจอ: dirty, busy, preview และ recovery

สถานะ	ความหมาย	ผลต่อการออกแบบ
draft	ข้อมูลที่กำลังแก้	อาจยังไม่ตรง DB
saved	id/version จาก DB	ใช้ส่งคำสั่งแก้ไข/ลบ
baseline	snapshot ที่บันทึกแล้ว	ใช้ตรวจการแก้ค้าง
busy / pending	งานยังไม่จบ	ป้องกันคำสั่งชน/ออกจากหน้า
ready	ตัวอย่างจัดหน้าเสร็จ	ไม่ใช่บันทึกสำเร็จ
overflow	เนื้อหาเกินที่จัดได้	เตือนและระงับ export ที่ไม่ปลอดภัย
recovery	สำเนากู้คืนแยก	ไม่ควรจองเลขหรือถือเป็นฉบับออกจริง

ป้องกันผลตอบกลับเก่าทับงานใหม่

Effect ที่สร้าง HTML ใช้ตัวแปร active และ cleanup เมื่อ draft เปลี่ยน งาน import/ประมวลผลเก่าที่กลับมาช้าไม่ควร setHtml แทนงานใหม่ ใช้หลักเดียวกันกับการค้นหาและโหลดไฟล์

Preview ส่งข้อความข้าม iframe ให้ตรวจ `event.source` ว่าเป็น frame ปัจจุบัน และตรวจรูปแบบ payload ก่อนเปลี่ยน state อย่ายอมรับข้อความจากทุกหน้าต่างเพียงเพราะชื่อ event ตรงกัน

กู้คืนกับบันทึกเป็นคนละกระบวนการ

`recovery.ts` เก็บ scope, sequence และ data เงื่อนไขอัปเดตยอมรับ sequence ใหม่กว่าเท่านั้น ป้องกัน autosave เก่ามาทับใหม่ เมื่อผู้ใช้กู้คืน ระบบเปิดข้อมูลเป็นร่างให้ตรวจและบันทึกเอง

กรณีทดสอบที่มีคุณค่า: พิมพ์ระหว่างบันทึก, เปลี่ยนเอกสารเร็ว, เปลี่ยนฟอร์มแล้ว preview เก่ากลับมาช้า, รีเฟรชหลังมีงานค้าง และอัปโหลดไฟล์ขณะกดออกจากหน้า

อ่านโค้ด: `src/components/workspace.tsx`, `src/components/recovery-panel.tsx`, `src/lib/recovery.ts`, `src/lib/preview-overlays.ts`

14

แบบข้อมูลและ SQLite: สิ่งที่ถูกเก็บจริง

`db()` สร้าง connection กลางผ่าน globalThis ใช้ node:sqlite DatabaseSync และ WAL พร้อม busy_timeout 5 วินาที เพื่อลดปัญหารอ lock ระยะสั้น โครงสร้างหลายตารางสร้างเมื่อมีการใช้งานโมดูลนั้นครั้งแรก

กลุ่มตาราง	เนื้อหา
documents / audit / document_numbers	ร่าง ประวัติการกระทำ และเลขที่จอง
document_attachments / document_appendix_images / signed_scans	ไฟล์แนบ รูป และไฟล์ลงนาม
correspondence_entries / correspondence_events	ทะเบียนรับ-ส่งและประวัติ snapshot
inventory_items / inventory_movements / inventory_events	รายการคงคลัง ใบรับ/จ่าย/ยืม และเหตุการณ์
inventory_files / inventory_form_drafts	รูป/หลักฐาน และร่างแบบฟอร์มพัสดุ
thai_dictionary / organization_directory	คำเฉพาะและข้อมูลหน่วยงาน
settings / recovery_drafts	ค่าระบบและข้อมูลกู้คืน

JSON กับคอลัมน์ปกติอยู่ร่วมกัน

ร่างเก็บเนื้อหาเป็น JSON ใน `documents.data` ส่วน id/version/updated_at เป็นคอลัมน์ที่ใช้ค้นและคุมการเขียน การ decode ใช้ documentSchema.parse อีกครั้ง ทำให้ default ของฟิลด์ใหม่ทำงานกับข้อมูลเดิม

ข้อดีคือเพิ่มช่องได้สะดวก แต่ query ขนาดใหญ่และการย้าย schema ต้องคิดเพิ่ม การ cast `as Row[]` หลัง query เป็นเพียงคำรับรองต่อ compiler ไม่ใช่การพิสูจน์ว่า SQL คืนข้อมูลถูกชนิด

ไฟล์หลายประเภทเก็บเป็น BLOB ใน DB ส่วนไฟล์ mirror/export เป็นสำเนาข้างนอก อย่าลบ BLOB เพราะเห็นไฟล์ใน output แล้วคิดว่าซ้ำโดยไม่ตรวจขั้นตอนกู้คืน

อ่านโค้ด: `src/lib/db.ts`, `src/lib/inventory-db.ts`, `src/lib/attachments.ts`, `src/lib/appendix.ts` • Node SQLite รุ่น 22.13 [R6]

15

Transaction, version และการกดซ้ำ

ปัญหาที่ optimistic concurrency แก้

หน้าต่าง A และ B เปิดร่าง version 3 เหมือนกัน A บันทึกสำเร็จเป็น 4 แล้ว B ส่งข้อมูล version 3 มา หาก UPDATE ด้วย id อย่างเดียว งานของ A จะถูกทับ ระบบจึงใส่ version ใน WHERE

```sql
UPDATE documents
SET data = ?, version = version + 1, updated_at = ?
WHERE id = ? AND version = ? AND deleted_at IS NULL;
```

ถ้า changes เป็นศูนย์ต้องแจ้ง conflict แล้วให้โหลดข้อมูลล่าสุด ห้ามลองเขียนซ้ำด้วย version ใหม่เอง เพราะเท่ากับยอมทับสิ่งที่ผู้ใช้ยังไม่ได้อ่าน

Transaction เก็บทุกอย่างหรือไม่เก็บเลย

การบันทึกร่างและ audit อยู่ใน BEGIN IMMEDIATE/COMMIT เดียวกัน หากขั้นใดล้มเหลวให้ ROLLBACK งานสต็อกใช้หลักเดียวกัน: ใบหนึ่งมีหลายรายการ ต้องไม่ตัดบางรายการแล้วรายการสุดท้ายไม่พอ

Idempotency ไม่เท่ากับปิดปุ่ม

เครือข่ายอาจตอบกลับหายแม้ server บันทึกสำเร็จแล้ว ผู้ใช้จึงกดซ้ำได้ `createMovement` ใช้ id เดิมและตรวจ payload เดิมเพื่อคืนรายการที่สร้างแล้ว ส่วน transition ที่สถานะปลายทางเดิมจะคืนรายการโดยไม่ตัดยอดซ้ำ

สิ่งที่เพิ่ม	ต้องคิดคู่กัน
คำสั่งแก้ไข	expected version และ conflict message
คำสั่งสร้างจากปุ่มยืนยัน	request id คงเดิมเมื่อ retry
คำสั่งหลายรายการ	transaction และกรณี rollback
อัปโหลด Drive	identity/hash และสถานะ sync แยกจาก local save

อ่านโค้ด: `src/lib/db.ts` writeDraft, `src/lib/inventory-db.ts` createMovement/transitionMovement, `src/lib/correspondence-db.ts` saveRegister

16

เปลี่ยน schema โดยยังเปิดข้อมูลเก่าได้

เพิ่มฟิลด์ใน JSON

หากเป็นหมายเหตุเพิ่มเติมที่ร่างเก่าไม่มี ใช้ default ว่างและคงการแสดงผลเดิมเมื่อว่าง เหมาะกับข้อมูล optional ที่ไม่กระทบความหมายของเอกสารเก่า

```ts
// ตัวอย่าง patch ใน documentSchema; ยังไม่ได้เพิ่มให้ระบบ
contactPerson: z.string().trim().max(200).default(''),
```

การ rename ฟิลด์หรือลบ enum value ทำให้ parse ข้อมูลเดิมไม่ผ่านได้ ก่อนเปลี่ยนต้องสำรวจตัวอย่างข้อมูลที่ลบข้อมูลระบุตัวตนแล้ว ออกแบบการอ่านของเก่า และทดสอบ backup เก่าด้วย

เพิ่มตารางหรือคอลัมน์

ปัจจุบันใช้ CREATE TABLE IF NOT EXISTS และมีการตรวจคอลัมน์ก่อน ALTER เช่น deleted_at ยังไม่มี migration framework ที่มีเลขรุ่นรวมทั้งระบบ อย่าสมมติว่าการ rollback โปรแกรมจะย้อน DB ได้เสมอ

ขั้นตอนที่ควรทำ:

1. ระบุ schema ก่อน/หลัง และวิธีแปลงค่าที่มีอยู่
2. สำรอง DB ด้วยวิธีของระบบ แล้วทดสอบกับสำเนา
3. migration ต้องรันซ้ำได้ ไม่สร้างข้อมูลซ้ำหรือหาย
4. ตรวจจำนวนรายการ ตัวอย่างค่าขอบเขต และ integrity_check
5. ปรับ exporter/restore validator และทดสอบคืนข้อมูล
6. ระบุการย้อนกลับว่าใช้ reverse migration หรือคืน backup

จุดพลาดสำคัญ: ตารางใหม่ไม่เข้าสำรองอัตโนมัติเพียงเพราะสร้างใน DB ต้องเพิ่มใน allowlist ของ `scripts/workspace-backup.mjs` และ trusted schema/validation ใน `src/lib/workspace-restore.ts` ด้วย

อ่านโค้ด: `src/lib/db.ts`, `scripts/workspace-backup.mjs`, `src/lib/workspace-restore.ts`, `tests/workspace-restore.test.ts`

17

รูปแบบข้อความ: ไม่ใช่ HTML editor เต็มรูปแบบ

BodyEditor ใช้ textarea เก็บข้อความ ตรรกะทำตัวหนาใส่ `**...**` รอบข้อความที่เลือก ตารางใช้รูปแบบ pipe แบบจำกัด แล้ว renderer แปลงเป็น HTML โดย escape ข้อมูลก่อน

```
TEXT

ข้อความย่อหน้าแรก มี **ข้อความตัวหนา** ในประโยค
ข้อความย่อหน้าใหม่
| รายการ | จำนวน |
| --- | --- |
| กระดาษ | 2 |
```

ทุก newline ในเนื้อหาทั่วไปหมายถึงเริ่มย่อหน้าใหม่ ไม่ใช่การบังคับตัดบรรทัดตามหน้าตัวอย่าง ข้อความย่อหน้าหนึ่งควรต่อเนื่องให้ตัวจัดหน้าตัดเอง ห้ามบันทึก newline ทุกบรรทัดที่ Chromium แสดง

ลำดับเนื้อหา structured ในรุ่นนี้

`bodySections()` ส่งออก **เรื่องเดิม ⟶ ข้อเท็จจริง ⟶ ข้อระเบียบ/กฎหมาย ⟶ ข้อพิจารณา ⟶ ข้อความเสนอท้าย** โดย proposal ไม่มีหัวข้อ หัวข้อ structured ใช้ขีดเส้นใต้และย่อหน้า 25 มม.

ถ้าต้องเปลี่ยนชื่อหรือลำดับ ต้องดูทั้งตัวกรอกใน document-tools และ bodySections ไม่แก้เพียงข้อความป้ายบน UI แล้วคิดว่าเอกสารจะเรียงตาม

ข้อจำกัด parser ที่ต้องรู้

ตารางแบ่ง cell ด้วยเครื่องหมาย | จึงไม่ใช่ Markdown ทั้งมาตรฐาน ไม่มี merged cells หรือ nested table อัตโนมัติ หากจะเพิ่ม ต้องกำหนดรูปแบบข้อมูลที่เก็บให้แน่นอนและให้ Word เข้าใจด้วย

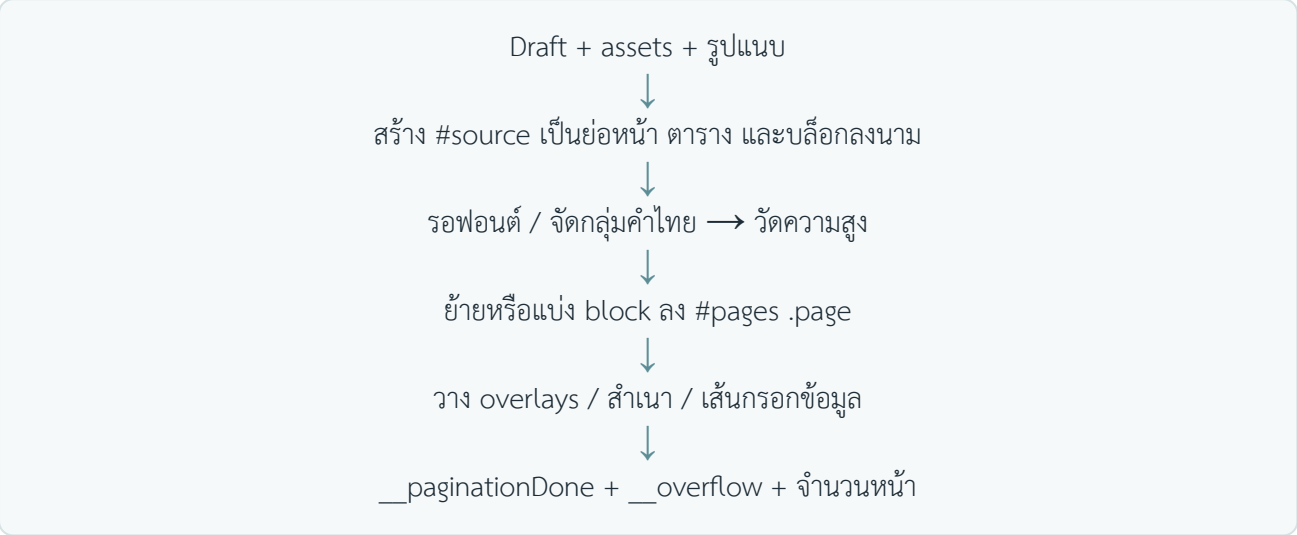
`contentBlocks` เป็นข้อมูลโครงสร้างอีกทาง เช่น เอกสารโอนงบ หากไม่ว่างอาจมีลำดับความสำคัญเหนือ bodySections จึงต้องตามเงื่อนไขใน documentHtml ก่อนสรุปว่าฟิลด์ body ไม่ทำงาน

อ่านโค้ด: `src/components/body-editor.tsx`, `src/lib/document.ts` bodySections, `src/lib/render-document.ts` formattedLines/body

18

ตัวจัดหน้า: จาก block ไปเป็นกระดาษ A4

`documentHtml()` สร้าง HTML พร้อมฟอนต์ รูป และ script จัดหน้า Browser เป็นผู้วัดความสูงจริงหลังฟอนต์พร้อม ไม่คำนวณความยาวจากจำนวนตัวอักษรอย่างเดียว



สิ่งที่ต้องรักษาเมื่อแก้

- ย่อหน้าที่ยาวต้องแบ่งหน้าโดยไม่สูญเสียข้อความ/ตัวหนา
- ตารางข้ามหน้าต้องมีหัวตารางซ้ำ และไม่ปล่อยแถวสูงเกินหน้าหายเงียบ ๆ
- ส่วนลงนามต้องพยายามอยู่ร่วมกัน โดยไม่ย่อทุกอย่างจนผิดขนาดตัวอักษร
- หน้าสำเนาและต้นฉบับต้องมีเลขหน้า/overlay ตามขอบเขตที่ตั้งใจ
- สัญญาณพร้อมต้องส่งหลังจัดเสร็จ ไม่ใช้เวลารอคงที่แทน

`options.wordParagraphs` เปิดโหมดรักษาย่อหน้าตรรกะสำหรับ Word ต่างจากการแบ่งบรรทัดใน PDF การแก้ paginator จึงต้องทดสอบทั้งสองโหมด

ปัญหา “ชื่อผู้ลงนามหลุดไปหน้า 2” อาจมาจากข้อมูลยาว ระยะขอบ ความสูงฟอนต์ หรือการรวม block ต้องวัดต้นเหตุ ไม่เพิ่มค่าติดลบเพื่อดันขึ้นในตัวอย่างเดียว

อ่านโค้ด: `src/lib/render-document.ts`, `src/lib/thai-layout.ts` • ทดสอบ: `tests/e2e/pagination-layout.spec.ts`, `tests/e2e/print-safety.spec.ts`

19

เรขาคณิตเอกสารและบทเรียนจากรุ่น 1.4.1

กลางกระดาษกับกลางพื้นที่ข้อความไม่เท่ากัน

A4 กว้าง 210 มม. ระยะซ้าย 30 มม. ขวา 20 มม. พื้นที่ข้อความกว้าง 160 มม. จุดกลางของพื้นที่ข้อความคือ 30 + 160/2 = **110 มม.** แต่กลางกระดาษคือ **105 มม.** จึงเยื้องขวา 5 มม.

การแก้รุ่น 1.4.0 ใช้ `--center-inset:10mm` กับขอบด้านขวาของบล็อกที่จัดกึ่งกลาง ทำให้ศูนย์กลางเลื่อนซ้ายครึ่งหนึ่งคือ 5 มม. ครุฑและหัวเรื่องจึงใช้แกนเดียวกัน แบบบันทึกข้อความมีครุฑด้านซ้ายตามรูปแบบของมัน ไม่ต้องจับครุฑทุกชนิดเข้ากลาง

เส้นประที่ขนาดไม่เท่ากัน

เส้นใต้ช่อง memo เดิมเป็น CSS dotted border แต่เส้นลงชื่อเป็นตัวอักษรจุดของ TH Sarabun New 16 pt รูปร่างและระยะจุดจึงต่างกัน แม้จะดูเป็น “เส้นประ” เหมือนกัน

รุ่น 1.4.1 เพิ่ม `.document-dot-guide` เป็นจุดของฟอนต์เดียวกัน วัดความกว้างจุดด้วย canvas แล้วเติมเท่าที่พื้นที่รับได้ เพิ่มหลังจัดหน้าและกำหนด aria-hidden เพื่อไม่ให้เป็นเนื้อหาหัวหนังสือ

Word extractor ข้ามจุดตอนอ่านข้อความหลัก แล้วส่งเป็น overlay แยก มิฉะนั้น “ส่วนราชการ” จะมีจุดติดเป็นข้อความซ้ำ

หน่วย	ใช้ที่ใด	การแปลงที่ใช้ใน Word
mm	กระดาษ/ตำแหน่งภาพ	1 นิ้ว = 25.4 มม.
CSS px	ผลวัดใน Chromium	96 px = 1 นิ้ว
pt	ขนาดอักษร	72 pt = 1 นิ้ว
twip / EMU	DOCX ระยะ/รูป	px × 15 / px × 9525

อ่านโค้ด: `src/lib/render-document.ts`, `src/lib/word-layout.ts`, `src/lib/word.ts` • ทดสอบ: header-alignment.spec.ts, dotted-guides.spec.ts

20

สร้าง PDF: ลำดับและการวินิจฉัย

`renderPdf(draft)` ตรวจรูปแนบ โหลด data URLs เปิด Chromium แบบ headless แล้วใส่ HTML ที่ได้จาก renderer รอ `__paginationDone` ตรวจ overlay ที่อ้างหน้าหายและ overflow ก่อนสั่ง page.pdf

```ts
// สรุปลำดับจาก pdf.ts ไม่ใช่ฟังก์ชันเต็ม
await page.setContent(documentHtml(draft, assets, sources));
await page.waitForFunction(() =>
  (window as unknown as { __paginationDone: boolean }).__paginationDone
);
const bytes = await page.pdf({
  format: 'A4', preferCSSPageSize: true,
  printBackground: true, displayHeaderFooter: false,
});
```

ระบบจริงยังมี validation, finally ปิด browser และรวมไฟล์แนบด้วย `appendAttachments()` จึงไม่ควรทำ route ใหม่ด้วยโค้ดตัดย่อนี้ตรง ๆ

แบ่งตรวจตามอาการ

อาการ	ตรวจที่จุดใดก่อน
เปิด browser ไม่ได้	executable path และสิทธิ์รัน Chrome/Edge
ค้างรอ pagination	script error, font load, block ที่แบ่งไม่ได้
ตำแหน่งภาพหาย	page ของ floatingItems และ missing overlay pages
preview ดี แต่ไฟล์แนบหาย	validateAttachments/readAttachment/appendAttachments
ฟอนต์ต่างจากตัวอย่าง	assets, public/fonts และ path ใน desktop build

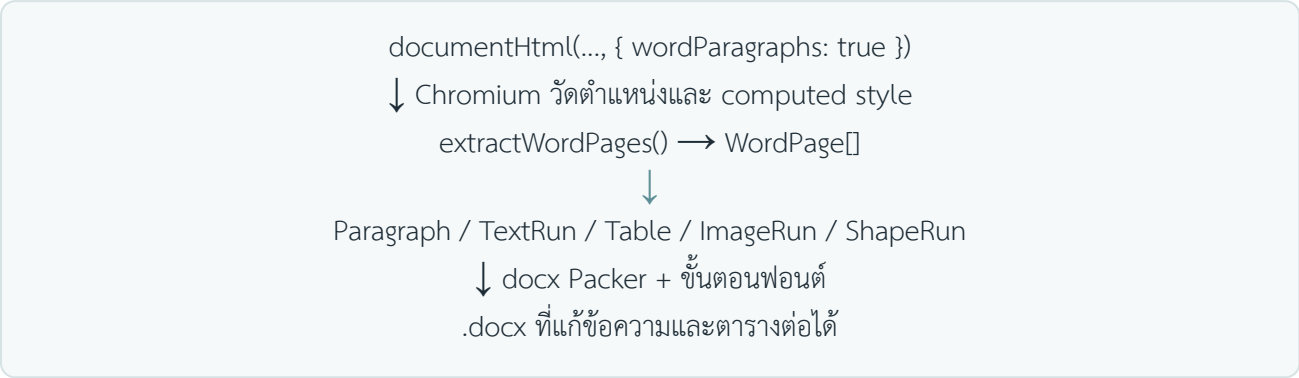
`page.route('**/*', abort)` ใน renderer ป้องกันการโหลดภายนอก เพราะ assets ถูกฝังไว้แล้ว ถ้าเพิ่มรูปผ่าน URL เฉย ๆ อาจขึ้นในเว็บแต่ไม่ขึ้นใน PDF ควรผ่านขั้นตอนรับไฟล์/ฝังรูปของระบบ

HTTP route ส่งออกอ่านร่างที่บันทึกใน DB ดังนั้น UI ต้องบันทึกค่าล่าสุดสำเร็จก่อนเรียก URL export มิฉะนั้นผู้ใช้จะเห็นไฟล์เวอร์ชันเก่า

อ่านโค้ด: `src/lib/pdf.ts`, `src/lib/attachments.ts`, `src/app/api/documents/[id]/pdf/route.ts`

21

Word: ใช้โครงสร้างแก้ไขได้ ไม่ใช่ถ่ายภาพทั้งฉบับ



`word-layout.ts` อ่าน DOM เป็นโครงสร้างกลาง ประกอบด้วย blocks ของย่อหน้า/ตาราง และ overlays ของข้อความ/รูป ส่วน `word.ts` แปลงโครงสร้างเป็น DOCX จึงควรแก้ปัญหาที่ชั้นถูกต้อง

ข้อความภาษาไทย

ย่อหน้าที่ align เป็น justify ถูกส่งเป็น `AlignmentType.THAI_DISTRIBUTE` รักษาหนึ่งย่อหน้าตรรกะ พร้อม run ของตัวหนา/ขีดเส้นใต้ และภาษา th-TH ส่วน `noProof:true` ปิดเส้นตรวจสะกดในข้อความส่งออก ไม่ได้พิสูจน์ว่าเขียนถูก

Word ใช้ตัวจัดหน้าของตัวเอง ระยะอาจต่างตามรุ่น Word ฟอนต์ และชุดภาษา ถึง XML ถูกก็ต้องตรวจด้วย Word จริงก่อนรับรองรูปลักษณ์ข้ามเครื่อง

ไฟล์แนบและขอบเขตแก้ไข

ข้อความ/ตารางที่ระบบสร้างแก้ใน Word ได้ ส่วน PDF ที่ผู้ใช้อัปโหลดแนบถูกแปลงเป็นภาพหน้ากระดาษใน DOCX ไม่ใช่ OCR และไม่คืนโครงสร้าง Word ต้นฉบับ ระบบยังไม่รองรับนำเข้า .docx กลับเป็น Draft อัตโนมัติ

เมื่อเพิ่มบล็อกใหม่ใน HTML ให้ตรวจว่า extractWordPages อ่านได้ หากเป็น class ใหม่ที่ไม่อยู่ใน selector อาจเห็นใน preview แต่หายจาก Word ได้

อ่านโค้ด: `src/lib/word-layout.ts`, `src/lib/word.ts` renderWord/renderWordHtml, `docs/WORD-EXPORT.md` • ทดสอบ: `tests/word.test.ts`, `tests/e2e/word-export.spec.ts`

22

เอกสารแนบสามชนิดที่ห้ามสับสน

ชนิดข้อมูล	หน้าจอ	ผลลัพธ์และแหล่งเก็บ
appendixPages	AppendixPagesEditor	หน้าข้อความที่สร้างในระบบ อยู่ใน JSON ร่าง
appendixImages	AppendixEditor	รูป+คำบรรยาย/ขนาด อ้าง BLOB รูปด้วย id
attachmentFiles	AttachmentEditor	PDF/ภาพที่อัปโหลด เก็บ metadata และไฟล์ใน DB

ภาพลอย/QR/ลายเซ็นใช้ `floatingItems` อีกกลุ่มหนึ่ง เพราะวางตามตำแหน่งหน้า ไม่ใช่ภาพที่ไหลในภาคผนวก

ข้อกำหนดจาก schema รุ่นนี้

ร่างมีหน้าข้อความแนบได้ถึง 20 รายการ ภาพแนบถึง 20 ภาพ และไฟล์แนบถึง 10 ไฟล์ ข้อจำกัดการรับไฟล์จริงยังมีใน actions/lib และเพดาน Server Actions 12 MB ของ Next อย่าเปลี่ยนเพียง max ใน schema แล้วถือว่ารับไฟล์ใหญ่ได้ทุกทาง

`showNameInDocument` คุมการแสดงชื่อในรายการเอกสารแนบ ไม่ได้ลบไฟล์ออกจากฉบับส่งออก `quantity/unit` เป็นข้อมูลรายการ ไม่ใช่จำนวนหน้า PDF

ถ้าเพิ่มชนิดไฟล์

1. ตรวจ MIME และข้อมูลไฟล์จริง ไม่เชื่อนามสกุลอย่างเดียว
2. กำหนดว่าจะ normalize เป็น PDF/ภาพอย่างไร และมีต้นฉบับให้ดาวน์โหลดหรือไม่
3. ปรับ schema, uploader, reader, preview และ export ทั้ง PDF/Word
4. กำหนดขนาดต่อไฟล์/ต่อคำขอ/ต่อร่างให้สอดคล้อง
5. ทดสอบไฟล์เสีย ไฟล์เกินขนาด metadata ไม่ตรง และสำรอง/กู้คืน

ห้ามเพิ่มการรับ HTML/SVG แบบแทรกตรงในเอกสารโดยไม่มีนโยบาย sanitize เพราะ renderer มี script จัดหน้าอยู่ภายใน

อ่านโค้ด: `src/lib/appendix.ts`, `src/lib/attachments.ts`, `src/app/attachment-actions.ts`, `src/app/appendix-actions.ts`, `src/lib/document.ts`

23

ภาพลายเซ็นและข้อความลอย: พิกัดต้องไม่ผูกกับซูม

ข้อมูล overlay เก็บ page แบบเริ่มที่ 1, xMm, yMm และ widthMm อ้างจากกระดาษ ไม่ใช่พิกเซลของจอ ผู้ใช้จึงซูม preview ได้โดยตำแหน่งจริงไม่เปลี่ยน

```
TEXT

พิกัดบนหน้าจอ → หักขอบกระดาษ → หารอัตราซูม
→ แปลง CSS px เป็น mm → clamp ในพื้นที่หน้า
→ บันทึกลง floatingItems
```

ตัวอย่าง: เลื่อนเมาส์ 40 CSS px ที่ซูม 50% เท่ากับ 80 px ในกระดาษ ก่อนคูณ 25.4/96 ถ้าข้ามขั้นหารซูม ภาพจะกระโดดหรือเลื่อนช้ากว่าเมาส์

สัญญาข้อมูล

ภาพลายเซ็นใช้ kind:image และ purpose:signature ไม่ใช่ PDF certificate signature ภาพยังคงเป็นรูปเหมือน QR หรือรูปประกอบ เพียงมีเครื่องมือเฉพาะช่วยวางตำแหน่ง

`preview-overlays.ts` จัดการการโต้ตอบใน iframe; `floating-item.ts` มีตรรกะประกอบ; Workspace รับและส่งสถานะเครื่องมือกลับเข้า iframe

กรณีที่ต้องทดสอบ

- วางบนหน้า 2 แล้วลดเนื้อหาจนเหลือหน้าเดียว: ต้องแจ้งหน้าที่หาย ไม่ทิ้งภาพเงียบ ๆ
- ลาก/ย่อขยายที่ซูม 50%, 100% และหน้าจอเล็ก
- กด Escape ออกจากโหมดวาง และเปลี่ยนเอกสารขณะเลือกภาพอยู่
- ส่งออกแล้วรูปอยู่ตำแหน่งเดียวกับ preview ทั้งต้นฉบับและสำเนา
- บันทึก เปิดใหม่ และสำรอง/กู้คืน แล้วรูปยังอ้างอิงได้

ตำแหน่งคงที่เป็นข้อจำกัดที่ตั้งใจ หากแก้เนื้อหาก่อนหน้า overlay จะไม่ติดตามคำเฉพาะเหมือน anchor ใน Word การเพิ่ม anchor-to-paragraph เป็นงานใหม่ที่ต้องออกแบบข้อมูลเพิ่ม

อ่านโค้ด: `src/lib/preview-overlays.ts`, `src/lib/floating-item.ts`, `src/components/signature-editor.tsx`, `tests/e2e/signature-placement.spec.ts`

24

คลังคำไทย: ตรวจคำกับกันแยกคำทำงานคนละส่วน

เส้นทางตรวจคำ

BodyEditor ⟶ ThaiProofreadPanel ⟶ checkThaiText ⟶ analyzeThai ใช้ `Intl.Segmenter('th', {granularity:'word'})` แบ่งคำ แล้วเทียบฐานคำ PyThaiNLP 62,106 คำและคำเฉพาะหน่วยงาน

ผลเป็น `ThaiIssue` มี word/start/end/suggestions/kind โดย kind เป็น suggestion หรือ unknown คำที่ไม่พบในคลังเป็นเพียงคำให้ทบทวน ไม่ใช่คำผิดแน่นอน ระบบจำกัดผลตรวจ 80 รายการและให้ผู้ใช้ตัดสินใจแก้

เส้นทางกันแยกคำ

`thaiKeepTerms()` รวมคำมาตรฐานกับคำเฉพาะ แล้ว `thai-layout.ts` ครอบช่วงข้อความเป็น `.thai-keep` ก่อนจัดหน้า โดยไม่ใส่ newline ลง Draft ส่วน Word ใช้ U+2060 ระหว่าง grapheme ในกลุ่มเดียวกัน

U+2060 เป็นอักขระไม่แสดงผล การคัดลอกจาก Word อาจพาตัวเชื่อมนี้ไปด้วย ควรระบุไว้เมื่อออกแบบนำเข้าหรือค้นข้อความในอนาคต

จุดปรับที่ชัดเจน

ต้องการ	แก้/ตรวจ
เพิ่มคำเฉพาะ	ตั้งค่าใน UI; ไม่ต้อง rebuild ฐานคำหลัก
เพิ่มคำผิดที่แนะนำ	commonThaiCorrections หรือรายการผู้ใช้
เปลี่ยนฐานคำหลัก	public/dictionaries, license/provenance, thaiLexicon
เปลี่ยนวิธีจับคำคู่	thaiKeepTerms + thai-layout + Word wordJoin
เพิ่ม grammar check	เป็นระบบใหม่ แยกจาก dictionary เดิม

ต้องตรวจคำยาวเกินคอลัมน์ ตัวหนาที่คั่นกลางคำ และย่อหน้าข้ามหน้า อย่าครอบทั้งประโยคเป็น nowrap เพื่อแก้คำเดียว เพราะจะทำให้ย่อหน้าล้นกระดาษ

อ่านโค้ด: `src/lib/thai-proofread.ts`, `src/lib/thai-dictionary.ts`, `src/lib/thai-layout.ts`, `docs/THAI-DICTIONARY.md`

25

ทะเบียนรับ-ส่ง: เลขทะเบียนและร่างเป็นคนละอย่าง

`correspondence-register.tsx` แสดงรูปแบบคล้ายเล่มทะเบียน รับค่าเป็น RegisterInput และบันทึกผ่าน correspondence-actions ไป saveRegister โดยมี actor, reason, requestId และ expected version

ส่วน	หน้าที่
RegisterEntry	เนื้อหารายการ serial/year/version และผู้ดำเนินการ
correspondence_events	snapshot ประวัติและเหตุผลการเปลี่ยน
request_id UNIQUE	กันคำสั่งสร้างซ้ำ
department_key/direction/year/serial UNIQUE	กันเลขลำดับทะเบียนซ้ำในขอบเขตเดียวกัน
documentId	เชื่อมกับร่างหนังสือโดยไม่รวมข้อมูลเป็นก้อนเดียว

แยกกฎปีให้ถูก

ทะเบียนรับ-ส่งมี year และลำดับของตัวเอง ส่วน document_numbers ใช้ปีจากวันที่หนังสือในการกันเลขซ้ำ ไม่ใช่เอาปีงบประมาณที่ใช้รายงานมาแทนทุกที่

ตัวกรองมีช่วงวัน ปีงบ และข้อความค้นหา normalize เลขไทย/อารบิก การเพิ่มตัวกรองต้องทำให้หน้าจอ สถิติ และ export ใช้เงื่อนไขเดียวกัน ไม่ให้ผู้ใช้เห็น 20 รายการแต่ export ออกไป 100 รายการโดยไม่ทราบ

ต่อไปถ้ามีหลายกอง/หลายผู้ใช้

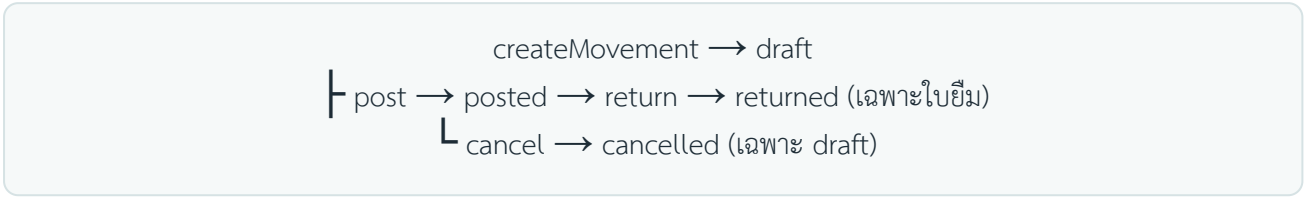
ชื่อ actor ที่กรอกยังไม่ใช่ user id ที่พิสูจน์ตัวตนแล้ว ถ้าต้องสอบย้อนกลับระดับระบบหลายคน ให้เพิ่มบัญชี สิทธิ์ แหล่งหน่วยงาน และ audit ที่ผูก session รวมถึงกำหนดว่าเลขทะเบียนมี namespace ใดบ้าง

ช่อง “เชื่อมร่าง” ใช้ draft-link-picker สำหรับค้นหา แทนการบังคับเลื่อน select ยาว เมื่อข้อมูลเพิ่มมากควรพิจารณา server pagination และ index จากการวัดจริง

อ่านโค้ด: `src/lib/correspondence.ts`, `src/lib/correspondence-db.ts`, `src/lib/correspondence-export.ts`, `src/components/draft-link-picker.tsx`

26

พัสดุ: state machine และความถูกต้องของยอด



ชนิด	เมื่อลงบัญชี	เงื่อนไขสำคัญ
receive	เพิ่ม balance	ครุภัณฑ์รายชิ้นรับซ้ำไม่ได้
issue	ลด balance	ใช้ supply และห้ามยอดติดลบ
loan	ลดจำนวนพร้อมใช้	ใช้ asset รายชิ้น จำนวน 1
return	เพิ่มจำนวนพร้อมใช้คืน	คืนครบใบที่ posted พร้อมสภาพเมื่อคืน

createMovement ยังไม่ตัดยอด การ post ต้องตรวจผู้สั่งจ่าย/ตำแหน่ง/หลักฐานสำหรับ issue และ loan แล้วบันทึกทุกบรรทัดใน transaction เดียว พร้อม event

balance ของครุภัณฑ์ขณะยืม หมายถึงจำนวนพร้อมใช้ ไม่ใช่จำนวนทรัพย์สินที่หน่วยงานเป็นเจ้าของ ดังนั้นรายงานสินทรัพย์คงเหลือไม่ควรหักทรัพย์สินที่ยืมไปเหมือนวัสดุเบิกใช้แล้ว

สิ่งที่เพิ่มไม่ได้ด้วยปุ่มอย่างเดียว

การคืนบางส่วนต้องเพิ่มข้อมูลคืนรายบรรทัด ตรวจยอดสะสมไม่เกินที่ยืม ปรับ state machine ประวัติ และรายงาน การยกเลิก posted ต้องมีกระบวนการกลับรายการ ไม่ลบ movement ทิ้งแล้วแก้ balance ตรง ๆ

การขาย/จำหน่าย การคิดค่าเสื่อม และ FIFO/เฉลี่ยยังไม่ใช่ระบบบัญชีครบวงจรในรุ่นนี้ แม้จะมีช่องให้กรอกในแบบฟอร์ม เอกสารที่กรอกได้ไม่เท่ากับระบบคำนวณและอนุมัติรายการนั้นแล้ว

อ่านโค้ด: `src/lib/inventory.ts`, `src/lib/inventory-db.ts`, `src/lib/inventory-validation.ts`, `tests/inventory.test.ts`

27

แบบฟอร์มพัสดุ 8 แบบ: นิยามข้อมูลนำหน้ารูปแบบ

id	แบบ
1	บัญชีวัสดุ
2	ทะเบียนคุมทรัพย์สิน
3	ทะเบียนคุมทรัพย์สินต่ำกว่าเกณฑ์
4	ใบเบิกพัสดุ
5	รายการวัสดุคงเหลือ
6	รายงานสินทรัพย์คงเหลือ
7	รายงานพัสดุชำรุดหรือไม่จำเป็นต้องใช้
8	รายงานการตรวจสอบพัสดุประจำปี

`inventoryForms` กำหนด fields, columns, roles และ landscape ตัว editor ใช้นิยามนี้สร้างส่วนกรอก ส่วน seedInventoryForm สร้างข้อมูลเริ่มต้นจาก inventorySnapshot ตามวันที่และรายการที่เลือก

ร่างแบบฟอร์มไม่ใช่สมุดบัญชี

ร่างอยู่ใน `inventory_form_drafts` มี version ของตัวเอง ผู้ใช้แก้แถวหรือราคาในร่างเพื่อจัดเอกสารได้ โดยไม่ไปเปลี่ยนยอดคลัง การตัดยอดต้องผ่าน movement เดิมเสมอ

`calculatedRows()` คำนวณ total ของแบบที่เกี่ยวข้องเฉพาะเมื่อจำนวน/ราคาพร้อม ไม่ควรเปลี่ยนค่าว่างเป็นศูนย์จนรายงานดูเสมือนมีราคาแล้ว

เพิ่มคอลัมน์หรือแบบใหม่

เริ่มที่ inventory-forms.ts แล้วตรวจ seed, calculatedRows, schema และเพดาน templateId ที่ปัจจุบันเป็น 1-8 ต่อด้วย renderer, UI, Word และ backup round-trip หากเพิ่มแบบ 9 แต่ไม่ขยาย schema จะเลือกได้แต่บันทึกไม่ผ่าน

รองรับร่างไม่เกิน 500 แถว แถวเดี่ยวที่สูงเกินหน้าต้องแจ้งให้ย่อ ไม่บีบฟอนต์จนอ่านไม่ได้ ตารางยาวต้องตรวจหัวตารางซ้ำและทุกแถวอยู่ครบ

อ่านโค้ด: `src/lib/inventory-forms.ts`, `src/components/inventory-form-workspace.tsx`, `src/lib/inventory-form-render.ts`, `src/lib/inventory-form-office.ts`

28

รายงานและตัวชี้วัด: รู้ว่าตัวเลขหมายถึงอะไร

`inventoryAnalytics(snapshot,start,end,today)` เป็นฟังก์ชันคำนวณหลัก แยกจากกราฟ จึงทดสอบสูตรได้โดยไม่เปิด UI รับ today เป็น argument ทำให้การทดสอบช่วงเวลาคงที่

ตัวชี้วัด	นิยามในรุ่นนี้
receives / issues	จำนวนใบที่ลงบัญชีในช่วง ไม่ใช่จำนวนชิ้น
receiveValue	ผลรวมจำนวน × ราคาที่บันทึกรับเข้า
changePercent	เทียบจำนวนใบเบิกกับช่วงก่อนหน้าที่ยาวเท่ากัน
coverage	floor(balance × days / จำนวนเบิกในช่วง) ต่อวัสดุ
low	วัสดุ active ที่ balance ≤ minimum
overdue	ใบยืม posted ที่ due น้อยกว่า today

หากช่วงก่อนมีศูนย์ใบ changePercent เป็น null เพราะคำนวณอัตราเพิ่มจากศูนย์ไม่ได้ หากไม่มีการเบิก coverage เป็น null ไม่ใช่ 0 วัน

ช่วงเวลาคนละความหมาย

รับเข้า/เบิกใช้ช่วงวันที่ที่เลือกและตัดวันสิ้นสุดที่วันนี้ ส่วนคงเหลือ/ใบยืมค้างเป็นสถานะปัจจุบัน ต้องแสดงวันที่อ้างอิงชัดเจน ห้ามติดป้ายทั้งหน้าว่า “ยอดสิ้นปี” เมื่อบางการ์ดเป็นยอดวันนี้

เพิ่ม metric อย่างไรให้เชื่อได้

เขียนนิยามก่อน เช่น lead time ใช้วันที่สั่งซื้อถึงรับจริงหรือถึงอนุมัติ แล้วตรวจว่ามีข้อมูลเหล่านั้นหรือยัง เพิ่ม pure calculation ⟶ unit test ⟶ UI label ⟶ export PDF/CSV ด้วยนิยามเดียวกัน

อย่ารวมจำนวนกระดาษ “รีม” กับเก้าอี้ “ตัว” เป็นจำนวนเดียว กราฟที่เหมาะอาจเป็นจำนวนใบหรือมูลค่าที่ฐานราคาครบ และต้องระบุข้อจำกัดของค่าเฉลี่ยเมื่อมีฤดูกาลหรือระยะจัดซื้อ

อ่านโค้ด: `src/lib/inventory-analytics.ts`, `src/components/inventory-analysis.tsx`, `src/lib/inventory-analysis-export.ts`, `src/components/reports-dashboard.tsx`

29

โมดูลอื่นที่ต้องรู้ก่อนเปลี่ยนระบบร่วม

โมดูล	จุดเริ่มอ่านและผลกระทบ
โอนงบ	budget.ts / budget-db.ts / budget-document.ts; สร้าง Draft ที่ใช้ contentBlocks เชื่อมรุ่นเอกสาร
งานธุรการ	office.ts / office-db.ts / office-workspace.tsx; ข้อมูลวัสดุเดิมไม่ใช่ยอดคลังใหม่โดยอัตโนมัติ
ห้องประชุม	booking.ts / booking-db.ts / meeting-bookings.tsx; ตรวจเวลาชนและข้อมูลห้อง
ปฏิทิน	calendar.ts / calendar-db.ts / work-calendar.tsx; วัน/เวลาและรายการนัด
คลังระเบียบ	legal.ts / legal-actions.ts / law-library.tsx; PDF และ metadata พร้อมลิงก์อ้างอิง
หน่วยงาน	organization-directory.ts / *-db.ts / organization-directory-manager.tsx; CRUD เพื่อเติมฟอร์ม
สแกนลงนาม	signed-scans.ts / signed-scan-actions.ts; แนบไฟล์กับ id ร่างและนับสถานะ

พาธในตารางอยู่ใต้ src/lib หรือ src/components ตามนามสกุล/บทบาท และ actions อยู่ src/app ใช้ดัชนีท้ายเล่มค้นชื่อเต็มได้

การเชื่อมข้อมูลควรชัดเจน

การกรอกหน่วยงานอัตโนมัติเป็นการคัดค่าลงร่าง ไม่ใช่ live link ที่เปลี่ยนเอกสารทุกฉบับเมื่อแก้ชื่อหน่วยงาน ส่วน link จากใบโอนงบไป Draft มีการติดตาม id/version ต้องอ่านเงื่อนไขก่อน regenerate

เมื่อเพิ่มฟิลด์ common เช่น เบอร์ติดต่อ ควรตัดสินใจว่าอยู่ใน directory ต้นแบบหรือในแต่ละร่าง หรือทั้งสองแบบ snapshot หากต้องการแก้เฉพาะฉบับ ไม่ควรไปเปลี่ยน directory ของทุกคน

ตรวจการเชื่อมต่อเมื่อซ่อนหรือลบโมดูล

เมนู ปุ่มค้นลัด รายงาน API ตารางสำรอง และลิงก์จากโมดูลอื่นอาจยังอ้างถึงของเดิม การลบ component อย่างเดียวไม่ใช่การถอนโมดูลครบถ้วน

อ่านโค้ด: `src/components/workspace.tsx`, `src/lib/storage.ts` reportData, `src/lib/action-search.ts`, `src/lib/db.ts` removeDraft

30

สำรอง กู้คืน และการเก็บรักษาข้อมูล

สำรองธรรมดาใช้ allowlist

`exportDatabase()` เปิด SQLite แบบอ่านอย่างเดียว อ่านชุดตารางใน transaction แล้วสร้าง DB สำรองใหม่ ตรวจ integrity และออก manifest/ไฟล์ประกอบ ไม่ได้คัดทั้ง settings หรือ .env แบบไม่เลือก จึงไม่ควรถือว่าชุดสำรองธรรมดารวม OAuth ไว้แล้ว

`workspace-restore.ts` ใช้ trusted schema ที่โปรแกรมกำหนด ไม่ใช้ SQL จากไฟล์อัปโหลดโดยตรง ตรวจข้อมูลก่อนนำกลับ ส่วนชุดอัปเดต/Personal มีขอบเขตการเก็บค่าต่างจากสำรองเอกสารทั่วไป

การสำรองตามรอบ

รอบที่ตั้งได้คือ 0 (ปิด), 6, 24, 168 ชั่วโมง เก็บ lastAttempt/lastSuccess/error และ lease กันงานซ้อน ทำงานเมื่อ server เปิดอยู่ ไม่ใช่ Windows Task Scheduler ที่ทำงานแม้โปรแกรมปิด

งานใหม่ต้องครบทั้งไปและกลับ

1. เพิ่มตารางใน exporter allowlist
2. เพิ่ม schema และตัวตรวจใน restore รวม foreign reference เชิงตรรกะ
3. ถ้ามี BLOB/ไฟล์ ให้กำหนด checksum และ metadata
4. ทดสอบรอบสร้าง ⟶ backup ⟶ เครื่องทดสอบว่าง ⟶ restore ⟶ export เอกสาร
5. ทดสอบไฟล์เสีย รุ่นเก่า และ id ที่อ้างถึงข้อมูลไม่พบ

สำรองอย่างเดียวไม่พิสูจน์ว่ากู้คืนได้ อย่าทดสอบ restore ทับ DB ของผู้ใช้ ควรใช้ fixture หรือชุดติดตั้งสำเนาที่ระบุชัดเจน

การคัดลอกเฉพาะ sarabun.sqlite ระหว่าง WAL ยังทำงานอาจไม่สะท้อนข้อมูลล่าสุดครบ ใช้ขั้นตอนสำรองของระบบหรือหยุดระบบและทำตามคู่มือกู้คืนที่ตรวจแล้ว

อ่านโค้ด: `src/lib/automatic-backup.ts`, `scripts/backup-worker.mjs`, `scripts/workspace-backup.mjs`, `src/lib/workspace-restore.ts`, `docs/BACKUP-RESTORE.md`

31

Drive และสิทธิ์: ขอบเขตของเครื่องเดียว

ปัจจุบัน `loginRequired()` คืน true เมื่อ SARABUN_REQUIRE_LOGIN=true เท่านั้น ค่าเริ่มต้นของงานนี้เปิดเข้าใช้ได้ทันทีผ่าน loopback ถ้าจะเปิดให้เครื่องอื่นเข้า ต้องออกแบบ authentication/authorization ใหม่ ไม่ใช่เปลี่ยน hostname เป็น 0.0.0.0 อย่างเดียว

Google Drive

`google/connect` และ callback ทำ OAuth; `drive.ts` จัดการ token และ sync; `local-config.ts` รับไฟล์ OAuth Web Client และตรวจ redirect URI; `storage.ts` เก็บตำแหน่งโฟลเดอร์ที่เลือก

เอกสารมี driveFileId และ syncedVersion แยกจาก version ใน DB เพื่อบอกว่าไฟล์บน Drive ตามทันร่างที่บันทึกหรือไม่ การ sync สำเร็จของ version เก่าไม่ควรถูกแสดงว่าเป็นฉบับใหม่ล่าสุด

สิ่งที่ต้องรักษา

- SESSION_SECRET เดิมใช้กับการเข้ารหัสข้อมูลเชื่อมต่อ อย่าสุ่มใหม่ทับชุดใช้งาน
- อย่าส่ง OAuth JSON หรือ .env.local เข้า Git/issue/ภาพประกอบคู่มือ
- callback ต้องตรง APP_URL รวม localhost กับ 127.0.0.1 ซึ่งเป็นคนละ origin
- local save ต้องไม่ล้มทั้งงานเพียงเพราะ Drive ออฟไลน์ ออกแบบสถานะรอ sync ชัดเจน
- ไฟล์พัสดุใช้ identity/hash ช่วยไม่ให้การ retry สร้างไฟล์ซ้ำ ต้องรักษาเมื่อปรับโครงโฟลเดอร์

ภาพลงชื่อไม่ใช่การอนุมัติออนไลน์

ชื่อผู้ให้ความเห็น ชื่อผู้รับส่ง และภาพลายเซ็นที่เลือกเป็นข้อมูลที่เจ้าหน้าที่บันทึก การเพิ่มบัญชีหลายคนจำเป็นต้องมี user id, role, ขอบเขตหน่วยงาน, audit จาก session และนโยบายเข้าถึงไฟล์รายรายการ

อ่านโค้ด: `src/lib/auth.ts`, `src/lib/drive.ts`, `src/lib/local-config.ts`, `src/lib/inventory-files.ts`, `src/app/api/google/callback/route.ts`, `docs/GOOGLE-DRIVE.md`

32

ปรับ UX/UI โดยไม่ทำให้แบบพิมพ์เปลี่ยน

CSS ของพื้นที่ทำงานถูกนำเข้าตามลำดับใน layout.tsx: globals ⟶ system-theme ⟶ workspace-tokens ⟶ workspace-polish ⟶ notion-workspace ⟶ workspace-comfort ส่วนหลายโมดูลใช้ CSS Module และฟอร์มมี document-form.css เพิ่ม

ลำดับนี้สำคัญ เพราะ selector ที่น้ำหนักเท่ากันและโหลดทีหลังอาจทับค่าก่อนหน้า อย่าแก้ทุกอย่างด้วย !important จนไม่รู้ว่าค่าจริงมาจากไฟล์ใด

เลือกจุดแก้ตามหน้าที่

เป้าหมาย	เริ่มที่
เมนู/กลุ่มงาน/ชื่อทางลัด	workspace-navigation.tsx, action-search.ts
ฟอร์มเรียงจากบนลงล่าง	document-form.tsx, document-tools.tsx
spacing/สีพื้นที่ทำงาน	workspace-tokens.css และ stylesheet ที่ครอง selector
ตารางรายการเอกสาร	document-dashboard.tsx และ .module.css
กระดาษและงานพิมพ์	render-document.ts ไม่ใช่ globals.css

แก้ scroll ให้ถูกชั้น

ตรวจ outer window, main shell, form pane, paper-scroll และ iframe ก่อน ควรระบุว่าใครเป็นเจ้าของความสูงและ overflow ไม่ใส่ height:100vh ซ้อนหลายชั้นจนมี scrollbar เกินจำเป็น

เวลา zoom กระดาษ wrapper ต้องมี width/height ตามขนาดหลัง scale ส่วน iframe ใช้ขนาดก่อน scale ถ้าสองส่วนไม่ตรงจะเกิดพื้นที่ว่างหรือเลื่อนแล้วกระโดด

ทดสอบคีย์บอร์ด focus, label, dialog, 320-1440 px, ซูมตัวหนังสือ และ prefers-reduced-motion ด้วย ทางลัดเลื่อนสมูทต้องยังเคารพผู้ใช้ที่ลดการเคลื่อนไหว

อ่านโค้ด: `src/app/layout.tsx`, `src/components/workspace.tsx`, `src/components/workspace-navigation.tsx` • ทดสอบ: form-scroll.spec.ts, workspace-comfort.spec.ts

33

Workshop: เพิ่มผู้ประสานงานในหนังสือภายนอก

ตัวอย่างฝึก ยังไม่ได้เพิ่มในโปรแกรมจริง เป้าหมายคือช่อง “ผู้ประสานงาน” ที่บันทึก/เปิดใหม่ได้ พิมพ์เฉพาะเมื่อกรอก และไม่ทำให้ร่างเก่าเสีย

1. กำหนดข้อมูลและ UI

เพิ่มใน documentSchema ที่ `src/lib/document.ts`:

```ts
contactPerson: z.string().trim().max(200).default(''),
```

ในส่วนท้ายหนังสือของ DocumentForm ใช้ helper field ที่มีอยู่และเงื่อนไข external:

```tsx
{external && field('contactPerson', 'ผู้ประสานงาน', { wide: true })}
```

type Draft มาจาก z.infer จึงมีฟิลด์นี้ตาม schema และ newDraft() ได้ default อัตโนมัติ ตรวจว่าตัวอย่าง/fixture ที่ประกาศ Draft แบบตรง ๆ ต้องเพิ่มค่านี้ด้วยหรือไม่จาก typecheck

2. ใส่ในเอกสารอย่างปลอดภัย

ใน helper contact() ของ render-document.ts เพิ่มเงื่อนไขก่อนจบ โดยใช้ e() ที่ escape/แปลงเลขตามแนวทางเดิม:

```ts
if (d.type === 'external' && d.contactPerson.trim()) {
  p(e('ผู้ประสานงาน ' + d.contactPerson), 'contact');
}
```

ใช้ p() ทำให้เป็นย่อหน้าปกติที่ Word extractor รองรับ แต่อย่าสรุปว่าทุก export จะถูกโดยไม่ทดสอบ ถ้าเพิ่ม custom HTML ที่ไม่ใช่ block เดิม ต้องตรวจ selector เพิ่ม

3. ความหมายของการเปลี่ยน

ไม่ต้องเพิ่ม SQL column เพราะฟิลด์นี้อยู่ใน JSON draft และ backup ของ documents รวม data อยู่แล้ว แต่ต้องทดสอบ parse backup/ร่างเก่าที่ไม่มีฟิลด์ และตำแหน่งเมื่อชื่อยาวหลายบรรทัด

อย่าใส่ required จนร่างเก่าทั้งหมดบันทึกไม่ได้ ถ้าต้องการบังคับก่อนออกจริง ให้เพิ่ม reviewIssues แยกจากข้อกำหนดเก็บร่าง

34

Workshop: พิสูจน์ว่าฟิลด์ใหม่ทำงานครบ

ใช้ไฟล์ทดสอบใหม่เฉพาะหลังทำ patch บท 33 โค้ดด้านล่างเป็นตัวอย่างทดสอบที่ต้องปรับ import ตามตำแหน่งไฟล์ ไม่ได้อยู่ในชุดทดสอบโปรแกรมปัจจุบัน

```ts
import { expect, it } from 'vitest';
import { documentSchema, newDraft } from '../src/lib/document';

it('keeps old draft data readable', () => {
  const old = { type: 'external', subject: 'Example' };
  expect(documentSchema.parse(old).contactPerson).toBe('');
});

it('accepts a contact but rejects an oversized value', () => {
  const base = newDraft('external');
  expect(documentSchema.parse({
    ...base, contactPerson: 'Officer A',
  }).contactPerson).toBe('Officer A');
  expect(documentSchema.safeParse({
    ...base, contactPerson: 'x'.repeat(201),
  }).success).toBe(false);
});
```

เกณฑ์ตรวจรับที่เหลือ

- กรอก ⟶ บันทึก ⟶ เปิดใหม่ แล้วชื่อยังอยู่ โดยใช้ DB ทดสอบ
- ค่า `<script>` แสดงเป็นข้อความ ไม่ทำงานเป็น script
- ไม่กรอกแล้วไม่มีบรรทัดว่างเพิ่มใน PDF
- ชื่อยาว/ภาษาไทย/ตัวเลข ยังไม่ชนส่วนลงนาม
- Word มีข้อความหนึ่งครั้งและแก้ได้ ไม่เป็นภาพทั้งบรรทัด
- backup ⟶ restore แล้วค่าอยู่ครบ; ร่างเก่ายังอ่านได้

ใช้รูปแบบ fixture จาก tests/db.test.ts และ Word test เดิม อย่าสร้าง test ที่เขียนลง data/sarabun.sqlite ของผู้ใช้

นิยามเสร็จ: ไม่ใช่เพียงเห็น input ใหม่ แต่ต้องเก็บ อ่าน แสดง ส่งออก และรักษาข้อมูลเก่าได้ หากข้อใดไม่ผ่านให้แก้ที่ชั้นนั้น ไม่ทำสำเนาค่าพิเศษเฉพาะ UI เพื่อหลบปัญหา

35

เพิ่มแบบหนังสือหรือเปลี่ยนรูปแบบการพิมพ์

เพิ่มประเภทหนังสือใหม่

1. เพิ่ม id/name/group ใน `templates.ts` และตรวจ DocumentType ที่ derive จากรายการ
2. ระบุว่าต้องมีฟิลด์ใดใน `documentSchema` พร้อม default สำหรับข้อมูลเก่า
3. เพิ่มเงื่อนไขฟอร์มใน document-form.tsx ให้เรียงตามหนังสือ
4. เพิ่ม branch ใน documentHtml รวมตำแหน่งหัวเรื่อง/วันที่/ลงนาม
5. เพิ่ม reviewIssues และ sample data ที่สมจริงแต่ไม่ใช่ข้อมูลผู้ใช้
6. ตรวจ gallery/search/report และรายการที่ hardcode เฉพาะบาง type
7. เพิ่ม lifecycle และ export tests ทั้ง short/long/table/attachments

อย่า reuse id เดิมเป็นหนังสือคนละชนิด เพราะร่างเก่าจะถูก render ด้วยความหมายใหม่ ถ้าจำเป็นต้องเปลี่ยนให้มี migration หรือเวอร์ชัน layout ที่อธิบายได้

เปลี่ยนระยะ/ฟอนต์ของแบบเดิม

การเปลี่ยน padding ของ .page กระทบทุกหน้า รวมวงจรแบ่งหน้า ลายเซ็น overlay และตำแหน่ง Word ควรเริ่มจาก rule เฉพาะชนิดเมื่อข้อกำหนดเฉพาะ ไม่เปลี่ยน global เพื่อแก้เอกสารตัวเดียว

หากเปลี่ยนรูปครุฑ ให้ตรวจ crop/viewport ด้วย เพราะไฟล์ต้นฉบับมีพื้นที่ขาว การกำหนด width กล่องรูปเท่ากันไม่ได้แปลว่าตัวครุฑมีขนาดเท่ากัน

Matrix ที่ต้องทดสอบ

ข้อมูล	สิ่งที่มักแตก
หัวเรื่องหลายบรรทัด	แนวกึ่งกลาง/ชนส่วนหัว
เนื้อหาเกือบเต็มหน้า	ลายเซ็นแยก/หน้าว่าง
ตารางหลายหน้า	header ซ้ำ/แถวหาย
ต้นฉบับและสำเนา	ตราผิดฉบับ/เลขหน้าผิด
Word	paragraph แตก/overlay หาย

อ่านโค้ด: `src/lib/templates.ts`, `src/lib/document-samples.ts`, `tests/e2e/all-form-lifecycle.spec.ts`, `tests/e2e/header-alignment.spec.ts`

36

ลดฟีเจอร์หรือซ่อนงาน: ไม่ทำข้อมูลเก่าหาย

แยก “ไม่ให้ใช้ต่อ” กับ “ลบข้อมูล”

หากต้องการให้ UI เรียบง่ายขึ้น อาจซ่อนทางเข้า หรือให้โมดูลเป็นอ่านอย่างเดียว โดยยังเปิดเอกสารเก่าและส่งออกได้ การลบตาราง/ไฟล์เป็นอีกการตัดสินใจหนึ่งและต้องมีแผนเก็บรักษา

งานที่ลด	จุดที่ต้องตรวจ
ซ่อนเมนู	WorkspaceView, navigation, action-search และ initialView
เลิกแบบหนังสือ	gallery แต่ยังอ่าน type เดิมและ render ได้
เลิกฟิลด์	นโยบายข้อมูลเดิม: แสดงต่อ/ย้าย/เก็บแต่ไม่แก้
เลิกโมดูลพัสดุ	reports, route, attachment และ backup ของข้อมูลเดิม
ปิด Drive	ปุ่มและงาน sync; local save/export ยังทำได้

แนวทางที่ปลอดภัยสำหรับเอกสารเดิม

เก็บ schema อ่านของเก่าไว้ แม้ UI ไม่สร้างรายการชนิดนั้นแล้ว หากถอน enum id ออกจาก schema ทันที listDrafts อาจ parse ไม่ผ่านเพราะมีร่างเก่าเพียงฉบับเดียว

ก่อนลบข้อมูลถาวรต้องระบุผลกระทบกับ audit เลขทะเบียน และเอกสารที่อ้าง id นั้น เช่น การทิ้งเลขที่จองแล้วอาจเปิดช่องให้ใช้เลขเดิมซ้ำ

เกณฑ์ตรวจรับ

ข้อมูลเก่ายังค้นเจอ/เปิดอ่านได้ ไม่มีเมนูค้างหรือทางลัดไปหน้าที่หาย รายงานยังนับอย่างมีนิยาม สำรอง/กู้คืนยังเก็บข้อมูลที่ต้องรักษา และมีคำอธิบายว่าฟีเจอร์ถูกปิดไม่ใช่ข้อมูลสูญหาย

การเก็บ field ไว้ใน schema แต่ไม่ render หมายความว่าไฟล์ export รุ่นใหม่จะต่างจากฉบับเก่า ต้องสื่อสารการเปลี่ยนนี้ด้วย โดยเฉพาะเอกสารที่เคยลงนามแล้ว

อ่านโค้ด: `src/components/workspace-navigation.tsx`, `src/components/workspace.tsx`, `src/lib/templates.ts`, `src/lib/action-search.ts`, `src/lib/workspace-restore.ts`

37

เพิ่มโมดูลใหม่โดยใช้รูปแบบเดิมของระบบ

แนวทางต่อยอด: สมมติจะเพิ่มงานซ่อมครุภัณฑ์ ไม่ใช่ฟีเจอร์ที่มีแล้วใน 1.4.1 เริ่มจาก lifecycle เช่น draft ⟶ submitted ⟶ completed พร้อมกฎว่าใคร/หลักฐานใดทำให้เปลี่ยนสถานะได้

ลำดับ	ไฟล์ที่เสนอ	หน้าที่
1	lib/repair.ts	schema, types, labels และกฎบริสุทธิ์
2	lib/repair-db.ts	CRUD/version/transaction/audit
3	app/repair-actions.ts	requireUser, parse และผลคำสั่ง
4	components/repair-workspace.tsx	ฟอร์ม รายการ ตัวกรอง และ dirty/busy
5	lib/repair-render.ts	preview และเอกสารออกแบบร่วม
6	api/repair/...	ดาวน์โหลดตาม id ที่ตรวจแล้ว
7	backup/restore + tests	เก็บและคืนข้อมูลทั้งชุด

เชื่อมครุภัณฑ์ด้วย id พร้อม snapshot ข้อมูลที่จำเป็นต่อเอกสารย้อนหลัง เช่น ชื่อ/รหัสตอนส่งซ่อม เพื่อไม่ให้การเปลี่ยนชื่อวันนี้ทำใบซ่อมเก่าเปลี่ยนความหมาย

อย่าคัดลอกทั้ง component แล้วเปลี่ยนชื่อ

เลือกใช้ชิ้นส่วนร่วม เช่น การเลือกเอกสาร แนบไฟล์ ผู้ลงนาม และตัวอย่าง A4 แต่แยกกฎ repair ออกจาก inventory movement การซ่อมอาจไม่ใช่การเบิกหรือลดยอด จึงต้องกำหนดสถานะพร้อมใช้อย่างชัดเจน

ก่อนเขียน UI

ตอบให้ได้ว่าความจริงของข้อมูลอยู่ที่ใด ใครแก้ได้ เมื่อใดถือว่ายืนยันแล้ว แก้ย้อนหลังได้หรือไม่ รายงานนับจากเหตุการณ์ไหน และยกเลิกอย่างไร จากนั้นเขียน unit test ของกฎที่มีความเสี่ยงก่อน เช่น ปิดงานซ่อมซ้ำและแก้ version เก่า

ใช้เป็นต้นแบบ: `src/lib/inventory.ts`, `src/lib/inventory-db.ts`, `src/app/inventory-actions.ts`, `src/lib/inventory-form-render.ts`

38

ทดสอบเป็นชั้นและใช้ข้อมูลแยกเสมอ

เลือกชุดตรวจตามผลกระทบ

แก้เรื่อง	ตรวจหลัก
type/schema/ฟังก์ชันคำนวณ	typecheck + unit test เฉพาะเรื่อง
CRUD/version/stock	DB tests ด้วย fixture ใหม่
ฟอร์ม/scroll/dirty	E2E ผ่าน isolated server
กระดาษ	layout E2E + ภาพที่ render จริง
Word	XML/text tests + เปิดใน Microsoft Word
DB schema/ไฟล์แนบ	backup/restore round-trip
launcher/update	desktop-update และ release tests

```powershell
npm run typecheck
npx vitest run tests/document.test.ts tests/word.test.ts
# หลัง build รุ่นที่ต้องการทดสอบแล้ว
npm run desktop:build
node scripts/test-release.mjs tests/e2e/word-export.spec.ts
```

`test-release.mjs` เปิดพอร์ต 3100 สร้างข้อมูลใหม่ใต้ output/v1-tests และไม่โหลด OAuth ของผู้ใช้ ต้องมี desktop build ล่าสุดก่อน มิฉะนั้นกำลังทดสอบซอร์สรุ่นก่อน

สำคัญ: playwright.config.ts มี baseURL เริ่มต้นที่พอร์ต 3000 คำสั่ง `npm run test:e2e` ตรง ๆ อาจยิงไปที่ระบบใช้งาน ให้ใช้ isolated runner สำหรับชุดที่มีการเขียนข้อมูล ส่วน test แบบ page.setContent ที่ไม่เรียกเว็บเป็นอีกกลุ่มหนึ่ง

หลักฐานกับข้อสรุป

รายงานผลควรมี commit, คำสั่ง, สภาพแวดล้อม, จำนวนผ่าน/ล้มเหลว และไฟล์ภาพ ข้อมูลเดิมของ 1.4.1 มีผล layout 7 กรณีและ Word 22 กรณีผ่าน ไม่ได้แปลว่าคู่มือนี้เพิ่งทดสอบทุกโมดูลใหม่ทั้งหมด

การสร้างไฟล์สำเร็จยังไม่พอ ต้องเปิดดูเนื้อหาและหน้าจริง โดยเฉพาะ Word ข้ามรุ่นและชุดโปรแกรมบนเครื่องที่สอง

อ่านโค้ด: `scripts/test-release.mjs`, `playwright.config.ts`, `vitest.config.ts`, `tests/db.test.ts`

39

ตารางแก้ปัญหา: ไล่จากอาการไปหลักฐาน

อาการ	ตรวจและแนวทาง
รันแล้วเจอ server เดิม	ตรวจ /api/health และ launcher state; ใช้ Stop-Sarabun.cmd ของชุดนั้น ไม่ฆ่า node ทุกตัว
dev ช้า แต่ desktop ปกติ	แยกเวลาติดตั้ง/build/first compile; ใช้ prebuilt สำหรับงานประจำ
preview ว่างเมื่อสลับร่าง	ตรวจ frameKey, ready, effect cleanup, postMessage source และ console ใน iframe
scroll กระโดด/ซ้อนหลายอัน	ตรวจความสูง form pane/paper-scroll และขนาด wrapper หลัง zoom
PDF เป็นข้อมูลเก่า	ตรวจ persist สำเร็จก่อนเรียก route และ id/version ที่อ่านจาก DB
Word ไม่เหมือน PDF	ตรวจ wordParagraphs, extractWordPages, run/paragraph และ font บนเครื่องเปิด
รูปหายเฉพาะ export	ตรวจ BLOB metadata/source mapping และการ block network ใน Chromium
ร่างเก่าเปิดไม่ได้	ตรวจ schema change, enum ที่ถอน และ default/migration
ใบเบิกยอดผิด	ตรวจสถานะ movement/event/transaction; ห้ามแก้ balance ตรงเพื่อให้ตัวเลขดูตรง
Drive เชื่อมไม่ได้	callback origin, client type, SESSION_SECRET เดิม และ error ที่ไม่เผย token
สำรองขาดข้อมูลใหม่	ตรวจ allowlist/schema/ไฟล์ประกอบใน export และ restore

วิธีบันทึกบัคให้คนต่อทำซ้ำได้

ระบุรุ่น ขั้นตอน ข้อมูลตัวอย่างที่ไม่เป็นความลับ ผลที่ควรได้ ผลที่พบ รูป/ไฟล์ตัวอย่าง และขอบเขตที่ทดสอบแล้ว เช่น “เฉพาะหัวข้อยาวสามบรรทัดในหนังสือภายนอกที่ซูม 75%” ชัดกว่า “ฟอร์มเพี้ยน”

แก้ต้นเหตุหนึ่งเรื่องแล้วรัน regression ที่เกี่ยวข้อง อย่าปรับค่า CSS หลายจุดพร้อมกันโดยไม่มีตัวอย่างก่อน/หลัง เพราะจะไม่รู้ว่าจุดไหนแก้และจุดไหนทำแบบอื่นเสีย

จุดหลักฐาน: output/desktop, output/v1-tests, Playwright screenshots และ health route; ตรวจความลับก่อนแชร์ log

40

Build, แจกโปรแกรม และ rollback

เส้นทางส่งมอบที่ควรทำ

1. เก็บงานบน branch ตรวจ diff และทดสอบเฉพาะผลกระทบก่อน
2. ปรับ version ใน package.json/package-lock และ CHANGELOG เมื่อเป็นรุ่นโปรแกรมใหม่
3. `npm run desktop:build` สร้าง standalone และชี้ current.json ไป build ใหม่
4. `node scripts/test-release.mjs ...` ทดสอบ build นั้นบน fixture แยก
5. สร้างแพ็กเกจอัปเดต ตรวจ hash และทดลองอัปเดตบนชุดสำเนา
6. เปิดโปรแกรม ตรวจ health สร้าง/เปิด/ส่งออกข้อมูลทดสอบ และลอง rollback ตามแผน
7. ส่งแพ็กเกจพร้อม release notes และสำรองที่คืนได้

```powershell
node scripts/update-desktop.mjs package releases/Update-NEW
# คำสั่งต่อไปใช้กับชุดสำเนาทดสอบก่อน ไม่ใช่เครื่องผู้ใช้
node scripts/update-desktop.mjs apply releases/Update-NEW
node scripts/update-desktop.mjs rollback --confirm-restore
```

เปลี่ยน Update-NEW เป็นชื่อโฟลเดอร์ใหม่ที่ยังไม่มี การ rollback คืนทั้งโปรแกรมและ snapshot ข้อมูลตามขั้นตอน จึงต้องเข้าใจว่าข้อมูลหลังอัปเดตอาจถูกแทนที่ ระบบมี rescue snapshot แต่ผู้ปฏิบัติงานยังต้องเลือกชุดให้ถูก

Full package กับ update package

`npm run desktop:export` สร้างชุดพกพาเต็ม ต้องหยุด server ตามเงื่อนไข script ก่อนตรวจ/export ส่วน update ใช้สำหรับชุดติดตั้งที่มีอยู่แล้ว ทั้งคู่ไม่ใช่แค่คัด src ไปให้ผู้ใช้

Build/runtime แยกจากข้อมูลธุรกิจ ตัว launcher เปิดจาก .desktop/versions และใช้ SARABUN_SOURCE_DIR อ้างกลับไปข้อมูล การแก้ src จึงไม่เปลี่ยนหน้าต่าง desktop ที่เปิดอยู่จนกว่าจะ build และเริ่มรุ่นใหม่

อ่านโค้ด: `scripts/build-desktop.mjs`, `scripts/desktop.mjs`, `scripts/export-desktop.mjs`, `scripts/update-desktop.mjs`, `QUICK-START-TH.md`

41

เส้นทางเรียนรู้และแบบฝึกพร้อมเกณฑ์ผ่าน

รอบที่ 1: อ่านข้อมูลให้เข้าใจ

เริ่ม `newDraft()` ⟶ documentSchema ⟶ bodySections เขียนฟังก์ชันรับ Draft แล้วคืนชื่อเรื่อง/วันที่ จากนั้นทดลองส่งค่าผิดชนิดและดู typecheck

ผ่านเมื่อ: อธิบายได้ว่า Draft ต่างจาก SavedDraft อย่างไร และเหตุใด JSON ต้อง parse แม้ประกาศ type แล้ว

รอบที่ 2: ทำ UI เล็ก ๆ

ทำ ContactField จากบท 12 ในชุดฝึก เปลี่ยนค่าโดย immutable update แล้วแสดงผลในหน้าจอ โดยยังไม่เชื่อม DB

ผ่านเมื่อ: พิมพ์ภาษาไทยได้ cursor ไม่หลุด และเปลี่ยนหนึ่งช่องไม่ล้างช่องอื่น

รอบที่ 3: ครบวงจรฟีเจอร์

ทำ workshop บท 33-34 เพิ่มผู้ประสานงาน เปิดร่างเก่าได้ บันทึก/เปิดใหม่ preview/PDF/Word ตรงกัน ใช้ fixture แยก

ผ่านเมื่อ: อธิบาย diff แต่ละไฟล์ได้และแสดงหลักฐาน export จริง ไม่ใช้แค่ screenshot input

รอบที่ 4: กฎธุรกิจและกรณีล้มเหลว

เพิ่ม test ของ version conflict และใบเบิกสองรายการที่รายการสุดท้ายคงเหลือไม่พอ ทดสอบให้ทั้งใบไม่ตัดยอด

ผ่านเมื่อ: ยอดเดิมคงอยู่หลังล้มเหลว และ retry ไม่ตัดซ้ำ

รอบที่ 5: ส่งงานให้คนอื่นดูแล

สร้างคู่มือเปลี่ยนแปลงสั้น ๆ ระบุข้อมูลที่เพิ่ม วิธีทดสอบ migration/backup และวิธีคืนรุ่น ทดลองกับชุด desktop สำเนา

ผ่านเมื่อ: คนที่ไม่ได้เขียนโค้ดทำตามขั้นตอนเปิดโปรแกรมและทดสอบฟีเจอร์ได้โดยไม่ถามชื่อไฟล์เพิ่ม

ไฟล์ฝึกแบบ runnable อยู่ใน `docs/developer-examples/` เป็นตัวอย่าง schema/union/generic ของเล่มนี้ ไม่แก้ฐานข้อมูลหรือเพิ่มฟีเจอร์ให้โปรแกรมจริง

42

หนี้ทางเทคนิคและลำดับปรับปรุงที่เหมาะสม

นี่คือข้อเสนอจากการอ่านโค้ด ไม่ใช่คำยืนยันว่าทุกรายการเป็นบัคที่กำลังเกิด และยังไม่ได้ลงมือเปลี่ยนผลิตภัณฑ์ระหว่างทำคู่มือ

ลำดับ	งานต่อยอด	เหตุผล / หลักฐานรับงาน
1	ทำเอกสารรุ่นและ test matrix ให้ตรงกัน	README บางหัวรุ่นยังเก่า; มีผลตรวจแยกหลายไฟล์ ควรรู้ว่าอะไรตรวจเมื่อไร
2	เพิ่ม migration ledger	schema ปัจจุบันกระจายในโมดูล; ต้องรู้รุ่น DB และรัน upgrade ซ้ำได้
3	แยก Workspace และ renderer เป็นส่วนย่อย	ไฟล์รวมหลายความรับผิดชอบ แยกโดยรักษา output tests เดิม
4	รวม CSS ownership	มี stylesheet ซ้อนหลายชั้น ลด override ที่ไล่ยากโดยคงหน้าตา
5	วัดข้อมูลปริมาณมาก	list/snapshot หลายส่วนโหลดทั้งชุด; ทดสอบหลักพันก่อนเลือก index/pagination
6	เพิ่ม visual regression ที่ควบคุมฟอนต์	แบบพิมพ์อ่อนไหวต่อช่องไฟ ต้องมี baseline ที่ไม่ใช้ข้อมูลจริง
7	ทดสอบเครื่องที่สอง/Word หลายรุ่น	เป็นหลักฐานคนละแบบกับ unit tests และ XML checks

หากจะเปิดใช้หลายคน

เป็นโครงการแยกที่ต้องเพิ่ม user/role/department, server deployment, audit จาก session, authorization ทุก API, concurrency หลายเครื่อง, backup กลาง และนโยบายเก็บไฟล์ ไม่ควรเริ่มจากแชร์ SQLite บน network drive

เลือกงานตามปัญหาจริง

เริ่มจากงานที่ลดความเสี่ยงข้อมูลและเพิ่มหลักฐานก่อนตกแต่งเพิ่ม ตั้งเกณฑ์สำเร็จที่วัดได้ เช่น เปิดรายการ 2,000 รายการภายในเวลาที่ทีมตกลง หรือ upgrade backup รุ่นเก่าแล้วเอกสารทุกชนิดยัง export ได้

ไม่ควร refactor ทั้งระบบพร้อมเพิ่มฟีเจอร์บัญชีใหม่ใน diff เดียว เพราะทดสอบและย้อนสาเหตุยาก

43

แหล่งอ้างอิงและดัชนีค้นเร็ว

ซอร์สหลักของเล่ม

ซอร์สตรึงที่ commit ce43948 เป็นแหล่งอ้างอิงพฤติกรรมทั้งหมดของระบบรุ่น 1.4.1 ชื่อไฟล์ใต้ “อ่านโค้ด” ในแต่ละบทสัมพันธ์กับราก repo นี้

ต้องการหา	ชื่อไฟล์/สัญลักษณ์
ช่องข้อมูล/ลำดับเนื้อหา	documentSchema / bodySections ใน document.ts
บันทึก/กันเขียนทับ	saveDocument / writeDraft / version
หน้า/ฟอนต์/ตราครุฑ	documentHtml / .page / .garuda
แปลง Word	extractWordPages / renderWord / renderWordHtml
คำไทย/กันแยกคำ	analyzeThai / thaiKeepTerms / wordJoin
รายงานพัสดุ	inventoryForms / seedInventoryForm / inventoryAnalytics
สำรอง/คืน	exportDatabase / workspace-restore.ts
ตัวเปิดโปรแกรม	desktop.mjs / current.json / build-desktop.mjs

เอกสารหลักสำหรับเรียนต่อ

- **[R1]** TypeScript: Everyday Types ใช้อ่านพื้นฐาน type และ inference
- **[R2]** TypeScript: Narrowing ใช้อ่านการตรวจเงื่อนไขเพื่อแยก union
- **[R3]** Zod: Basics ใช้อ่าน parse/safeParse; โปรเจกต์ตรึง Zod 4.5.4
- **[R4]** TypeScript: Generics ใช้อ่านการรักษาความสัมพันธ์ของ type
- **[R5]** TypeScript: Utility Types ใช้อ่าน Partial/Pick/Omit/Record
- **[R6]** Node.js 22.13: SQLite ใช้อ่าน DatabaseSync และ prepared statements ตามสายรุ่นที่ระบบใช้

เอกสารเว็บตรวจเมื่อ 24 ตุลาคม 2569 บางเว็บอัปเดตรุ่นใหม่ต่อเนื่อง ให้ยึด package-lock และคู่มือ Next ใน node_modules ของ checkout ประกอบเสมอ คำอธิบายในเล่มเรียบเรียงจากโค้ด ไม่ได้คัดลอกคู่มือภายนอกทั้งบท

44

เช็กลิสต์รับช่วงและนิยามงานเสร็จ

ก่อนเริ่มแก้

- ☐ ยืนยัน repo/branch/commit และ appVersion ที่เปิดใช้งานจริง
- ☐ แยก checkout, DB, พอร์ต และข้อมูลทดสอบจากผู้ใช้
- ☐ มี backup ที่ทดลองกู้คืนได้ และรู้ว่ารวม credentials หรือไม่
- ☐ อ่าน schema/action/DB/renderer ของงานที่จะเปลี่ยนครบเส้นทาง
- ☐ เขียนผลที่ต้องการและข้อมูลขอบเขต เช่น ยาว ขาดค่า และซ้ำ

ก่อนส่งโค้ด

- ☐ typecheck ผ่านและไม่มี any/cast ที่เพิ่มเพื่อกลบปัญหาโดยไม่อธิบาย
- ☐ unit/integration ผ่านตามกฎที่เปลี่ยน รวม conflict และ retry
- ☐ ฟอร์มบันทึก เปิดใหม่ และป้องกันออกจากหน้าขณะมีงานค้างได้
- ☐ preview/PDF/Word มีข้อมูลเดียวกัน ตรวจหน้าจริงและตัวอักษรครบ
- ☐ ร่างเก่า/backup เก่าเปิดได้ หรือมีแผน migration ที่ทดสอบแล้ว
- ☐ ตารางและไฟล์ใหม่รวมอยู่ใน backup/restore แล้ว
- ☐ Git diff ไม่มี .env, DB ผู้ใช้, token หรือไฟล์แนบส่วนตัว

ก่อนส่งโปรแกรมให้ใช้งาน

- ☐ desktop build ตรง commit ที่ทดสอบ และชุดอัปเดตตรวจ hash แล้ว
- ☐ ผ่าน isolated release test ไม่ใช้พอร์ต/DB งานจริง
- ☐ ตรวจ Clean/Personal และคู่มือเริ่มใช้ของเครื่องปลายทาง
- ☐ อธิบายข้อจำกัดที่ยังไม่ตรวจ เช่น Word อีกเวอร์ชันหรือเครื่องที่สอง
- ☐ มีบันทึกรุ่น สิ่งที่เปลี่ยน วิธีตรวจ และวิธีย้อนกลับ

ใช้เล่มนี้ต่ออย่างไร

ต้นฉบับ Markdown เป็นเอกสารที่แก้ต่อได้ สร้าง PDF ใหม่ด้วย `node scripts/build-developer-handbook.mjs` หลังติดตั้ง dependencies ตาม repo แล้ว ปรับเลขรุ่น/commit ในตัวสร้างและเนื้อหาเมื่อทำคู่มือรุ่นใหม่ อย่าเปลี่ยนเพียงปกแล้วถือว่าทั้งเล่มเป็นข้อมูลล่าสุด

การส่งต่องานที่ดีต้องทำให้คนถัดไปรู้ทั้ง “แก้ตรงไหน” และ “พิสูจน์อย่างไรว่ายังถูก” ใช้เช็กลิสต์นี้แนบกับ PR หรือ release note โดยเลือกข้อที่เกี่ยวข้องกับงานนั้นจริง